

Programmazione 1

Corso c

15/10/2023
> Array e Matrici

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica
Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Funzione Chiamante e Funzione Chiamata

Funzione che deve calcolare un **output scalare** con **input scalari** (es. `isOdd`).

Due possibilità:

1. La funzione chiamante passa l'input alla chiamata per **valore attraverso i parametri formali** e la chiamata passa l'output alla chiamante attraverso il **return**
2. La funzione chiamante passa l'input alla chiamata **per valore attraverso i parametri formali** e la chiamata passa l'output alla chiamante per **riferimento *indiretto/simulato* attraverso un parametro formale di tipo puntatore**

ATTENZIONE: nel caso 2 l'output deve essere per forza un puntatore!!!

Funzione Chiamante e Funzione Chiamata

- Perché l'input non viene passato per riferimento indiretto/simulato?

Golden Rule:

The usual rule is to pass quantities by value unless the program needs to alter the value, in which case you pass a pointer (CPrimerPlus)

Funzione Chiamante e Funzione Chiamata

Funzione che deve calcolare un **output vettoriale** con **input vettoriale** (es. `onlyOdd`).

Una sola possibilità:

1. La funzione chiamante passa il vettore in input alla chiamata per riferimento *simulato* attraverso 2 parametri formali (1p e `1int/size_t`) e la chiamata passa l'output alla chiamante per riferimento *simulato* attraverso 2 parametri formali (1p e `1int/size_t`)

Dove sono allocati i vettori? Perché?

Funzione Chiamante e Funzione Chiamata

- Un INPUT (vettoriale) può coincidere con l'OUTPUT (vettoriale)?
- Esempio:
 - Cubo
 - `scanf` (scalare)
- Non voglio permettere di far variare il parametro: `const`

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Filtri 1: IN vettore, OUT vettore

- `void filtroDispariIt(int a[], size_t aLen, int b[], size_t* bLen)` modifica b, memorizzandovi tutti e soli gli elementi di a il cui valore è dispari;
- `void filtroPosizioniPariIt(int a[], size_t aLen, int b[], size_t* bLen)` modifica b, memorizzandovi tutti e soli gli elementi di a che occupano posizioni pari;
- `void filtroPosizioniDispariElementiPariIt(int a[], size_t aLen, int b[], size_t* bLen)` modifica b, memorizzandovi tutti e soli gli elementi di a che occupano una posizione dispari avendo, simultaneamente, valore pari;

Attenzione alla signature!!!

- `void filtroDispariIt(int a[], size_t aLen, int b[], size_t* bLen);`
- `void filtroDispariIt(int a[], size_t aLen, int b[], size_t* bLen, size_t LMAX);`
- `int filtroDispariIt(int a[], size_t aLen, int b[], size_t LMAX);`

Filtri 2: IN vettore, OUT vettore

- `void filtroIntervalloIt (int a[], size_t aLen, int b[], size_t* bLen, int min, int max)` **modifica b**, memorizzandovi tutti e soli gli elementi di a il cui valore è compreso nell'intervallo [min, max], in cui gli estremi sono inclusi;
- `void filtroIndiciDoppioRiferimento (int a[], size_t aLen, int b[], size_t* bLen, int riferimento)` **modifica b**, memorizzandovi tutti e soli gli indici degli elementi di a che hanno valore doppio di riferimento.

multipliTre: IN vettori, OUT vettore

Scrivere una funzione C iterativa di nome `multipliTre` con le seguenti caratteristiche:

- `multipliTre` dipende dai seguenti parametri formali:
 - due array `a` e `b` di tipo intero, con dimensione massima `L_MAX > 0`, visti come input e tali che la parte effettivamente inizializzata sia delimitata dai valori in `aLen` e `bLen`, e un array `r` di tipo intero, con dimensione massima `L_MAX > 0`, visto come output;
- `multipliTre` restituisce `r` modificata come segue:
 - Per ogni indice `i` nell'intervallo `[0, minimo{aLen, bLen})`, in cui l'estremo inferiore è incluso e quello superiore escluso, l'elemento `r[i]` assume il valore `true` se almeno uno tra gli elementi di posizione `i` in `a` e `b` è multiplo di 3.
- `multipliTre` modifica anche il valore di `rLen` coerentemente con l'idea che un array, in questo caso `r`, è completamente descritto quando è noto il numero di elementi effettivamente utilizzati in esso.

IN array, OUT scalare

- Scrivere una funzione `C int sommaTuttiElementi(int a[], int la)` che, applicata ad un array `a`, restituisce la somma di tutti gli elementi in `a`.
- Scrivere una funzione `C int moltiplicaTuttiElementi(int a[], int la)` che, applicata ad un array `a`, restituisce la moltiplicazione di tutti gli elementi in `a`.
- Scrivere una funzione `C float sommaFrazioniTuttiElementi(int a[], int la)` che, applicata ad un array `a`, restituisce la somma di tutte le frazioni che si ottengono dividendo ogni elemento in `a` col successivo, ammesso che quest'ultimo esista.

CodeRunner

- Lab05-Es1 Somma selettiva
- Lab06-Es1 Statistiche sequenze

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Intervalli chiusi e semiaperti

- BinarySearch alla Deitel: $[0, n-1]$ vs. $[0, n)$
 - Left e Right potevano diventare negativi (sono size_t!!!)
- Possiamo riscrivere la binary search con un intervallo semiaperto? Quali sono i vantaggi?
- $[L, R)$
 - Il numero di elementi dell'intervallo è: $R - L$
 - L'intervallo è vuoto (non ha elementi) se: $R == L$
 - Se l'intervallo non è vuoto, allora:
 - L è il primo elemento
 - $R - 1$ è l'ultimo elemento

BinarySearch3

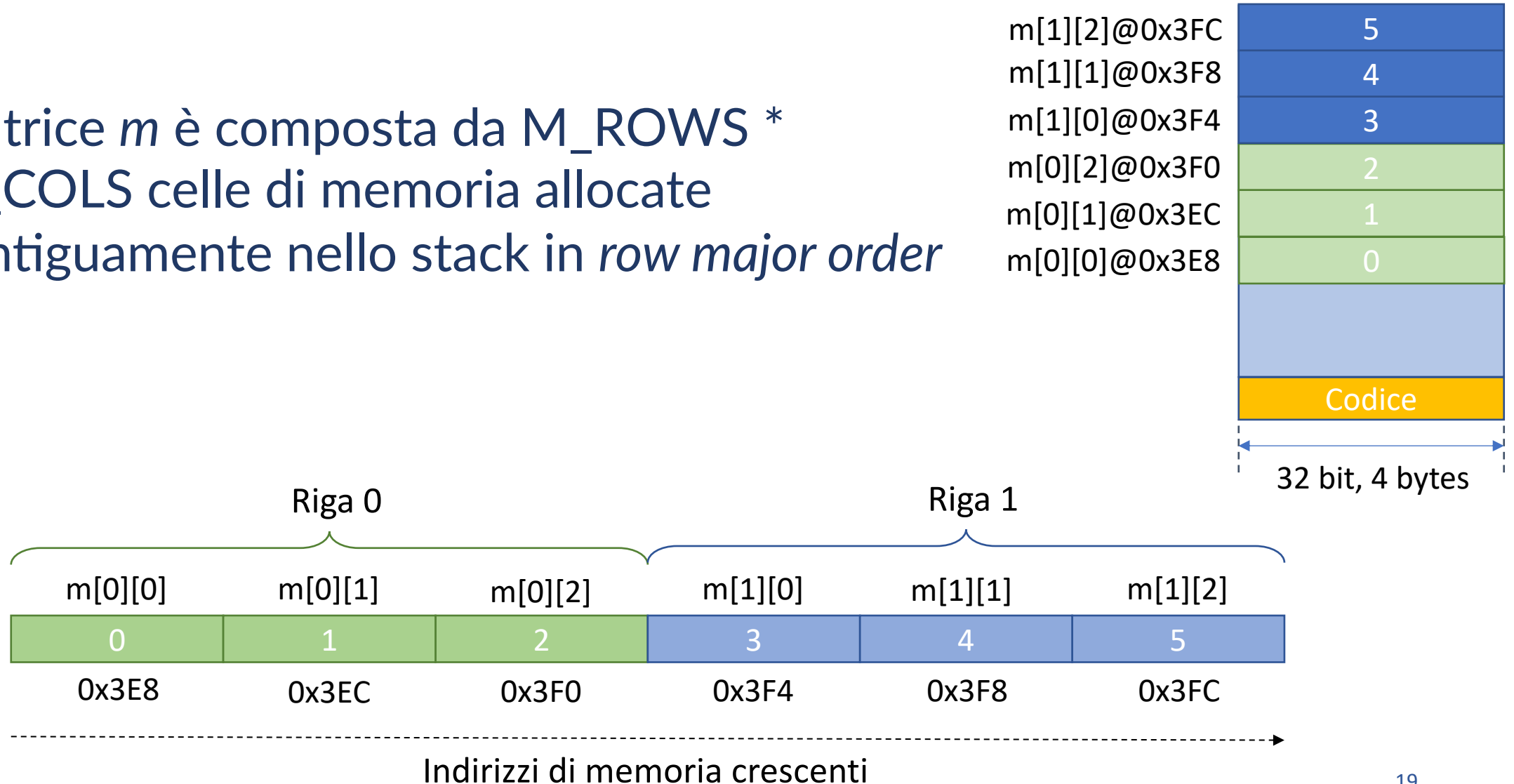
```
int binarySearch3(const int a[], int key, size_t left, size_t right) {
    int ret = -1;
    // loop until the interval [l, r) is not empty AND the key has not been found
    while (right - left > 0 && ret == -1) {
        size_t middle = (left + right) / 2; // determine middle subscript
        // display subarray used in this loop iteration
        printRow(a, left, middle, right);
        // if key matches, return middle subscript
        if (key == a[middle]) {
            ret = middle;
        }
        else if (key < a[middle]) { // if key < b[middle], adjust right
            right = middle; // next iteration searches left end of array
        }
        else { // key > b[middle], so adjust left
            left = middle+1; // next iteration searches right end of array
        }
    }
    // end while
    return ret;
}
```

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Modello di memoria

- Matrice m è composta da $M_ROWS * M_COLS$ celle di memoria allocate contigualmente nello stack in *row major order*

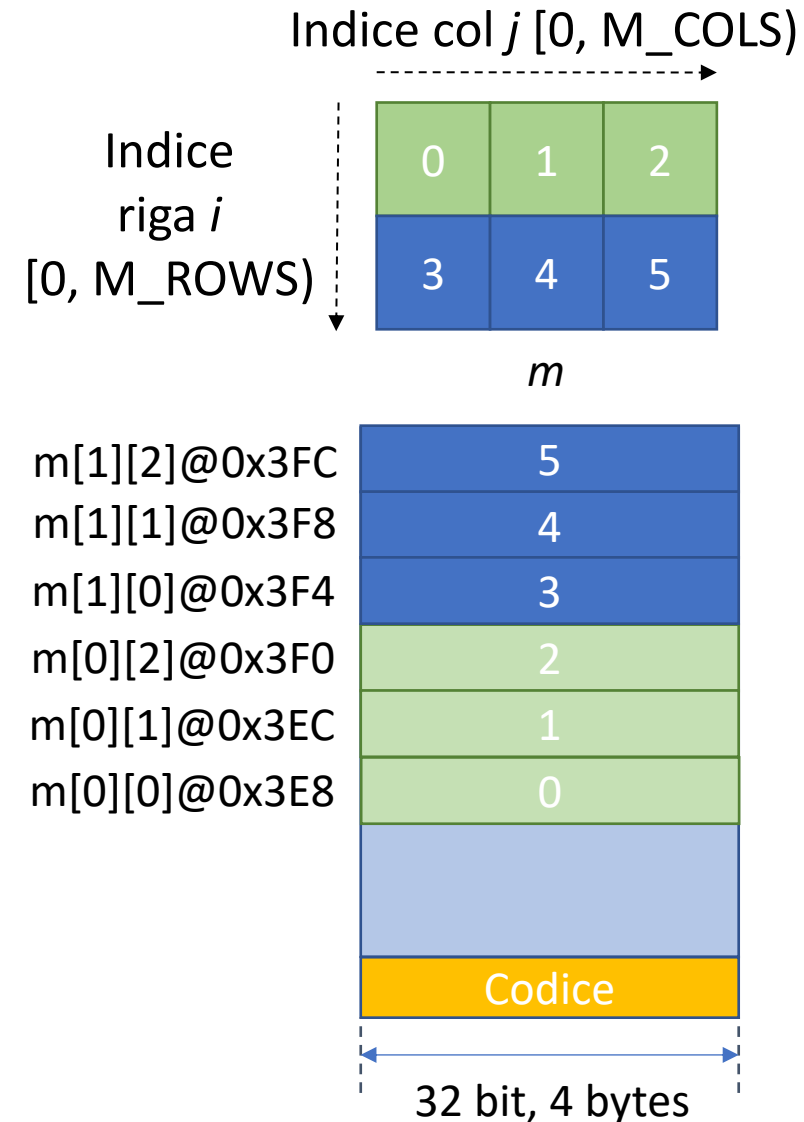


Modello di memoria

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2}, {3,4,5}};
    for (int i = 0; i < M_ROWS; i++) {
        for (int j = 0; j < M_COLS; j++) {
            printf ("m[%d][%d]@%p vale %d\n", i, j,
&m[i][j], m[i][j]);
        }
    }
}
```

```
m[0][0]@0x3E8 vale 0
m[0][1]@0x3EC vale 1
m[0][2]@0x3F0 vale 2
m[1][0]@0x3F4 vale 3
m[1][1]@0x3F8 vale 4
m[1][2]@0x3FC vale 5
```



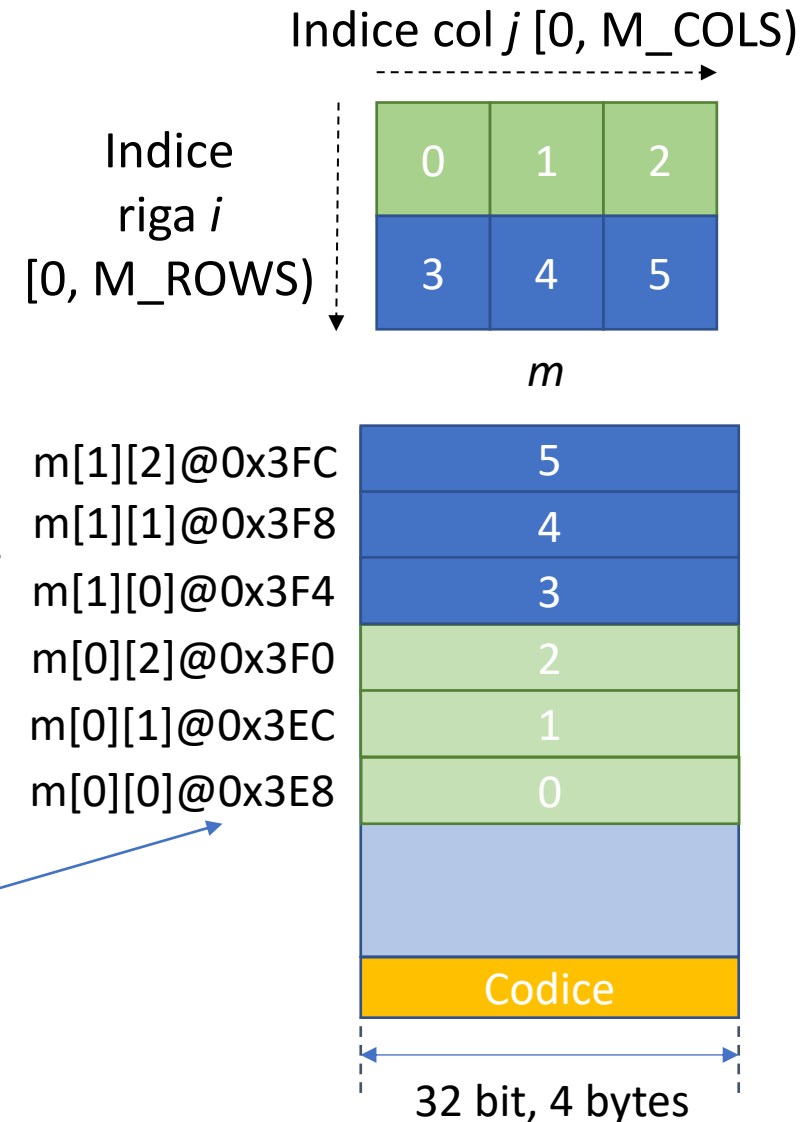
Puntatori a matrici

- Il nome della matrice è un puntatore alla cella di indirizzo più basso, ovvero `m[0][0]`

```
printf("m[0][0]@%p vale %d", m, *(int*)m);
```

`m[0][0] @ 0x3E8`

m è l'indirizzo di questa parola in memoria



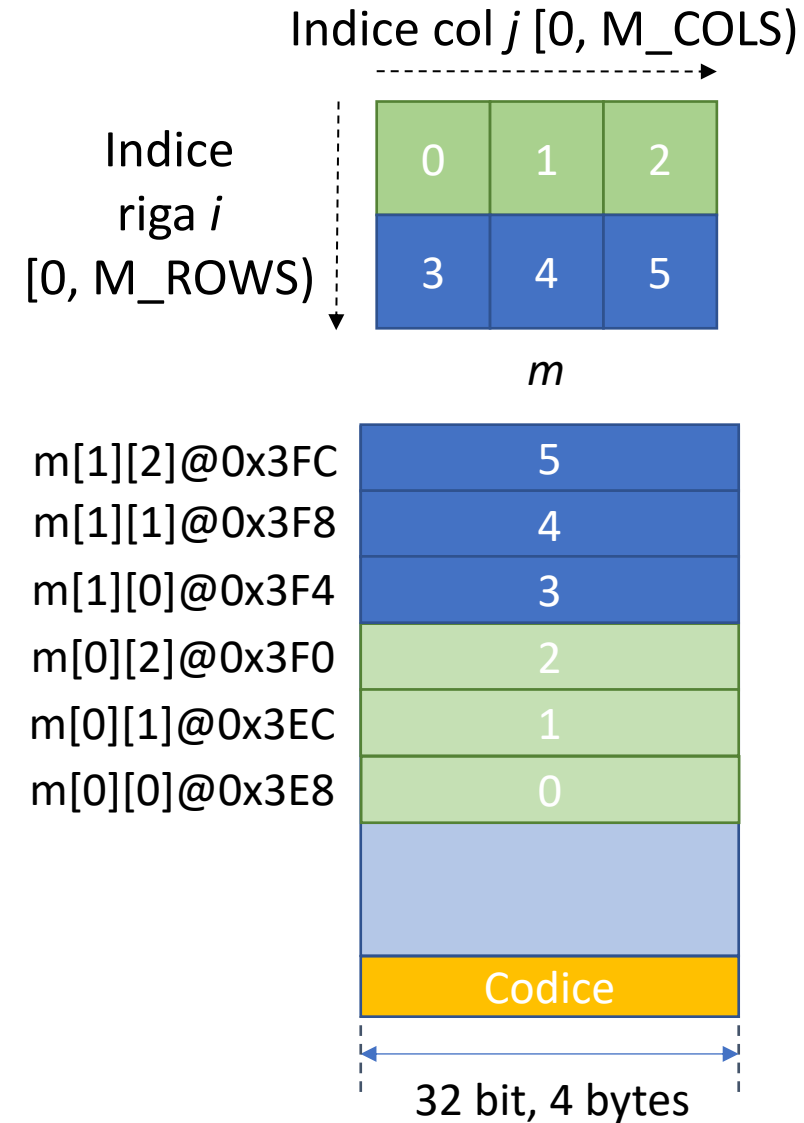
Puntatori a matrici

- Il nome della matrice è un puntatore alla cella di indirizzo più basso, ovvero `m[0][0]`

```
printf("m[0][0]@%p vale %d", m, * (int*)m);
```

`m[0][0] @ 0x3E8 vale 0`

Casting a `int*` di `m`
prima della
dereferenziazione



Puntatori a matrici

- Il nome della matrice è un puntatore alla cella di indirizzo più basso, ovvero `m[0][0]`

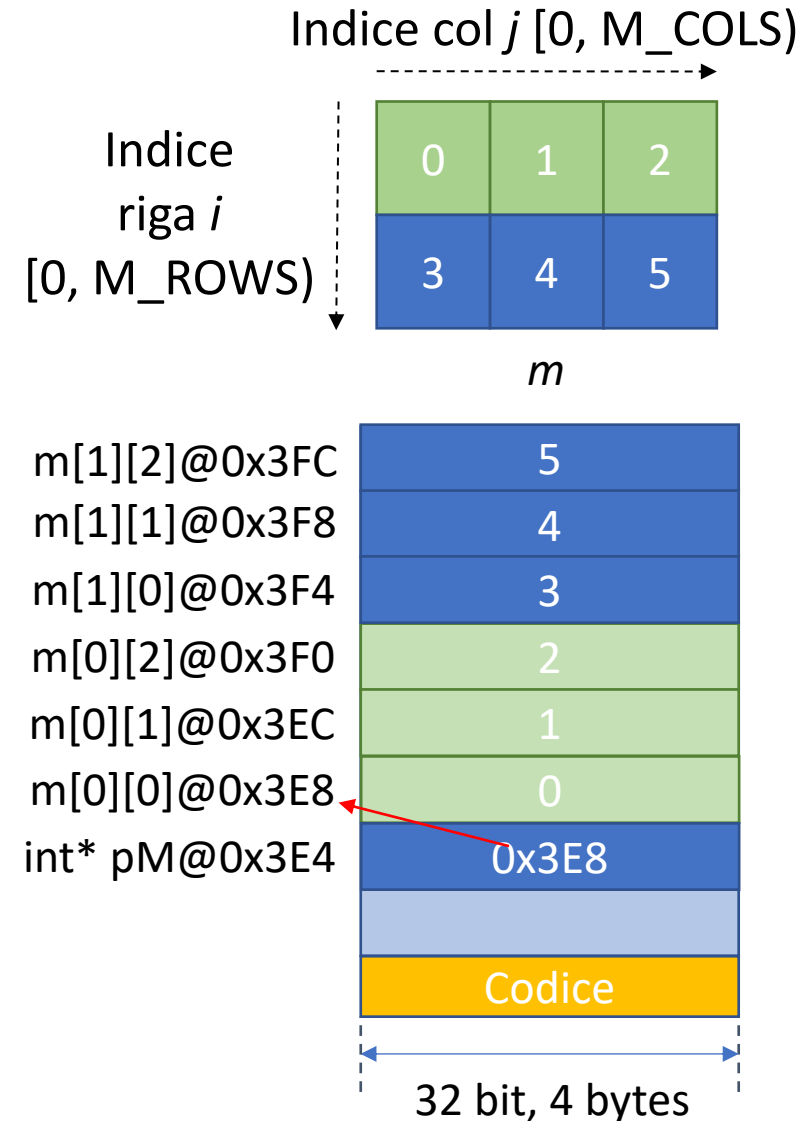
```
printf("m[0][0]@%p vale %d",m, *(int*)m);
```

`m[0][0] @ 0x3E8 vale 0`

- Si può ciclare sugli elementi di `m` dato un puntatore di lavoro (e opportuno casting)

```
int* pM = (int*) m;
```

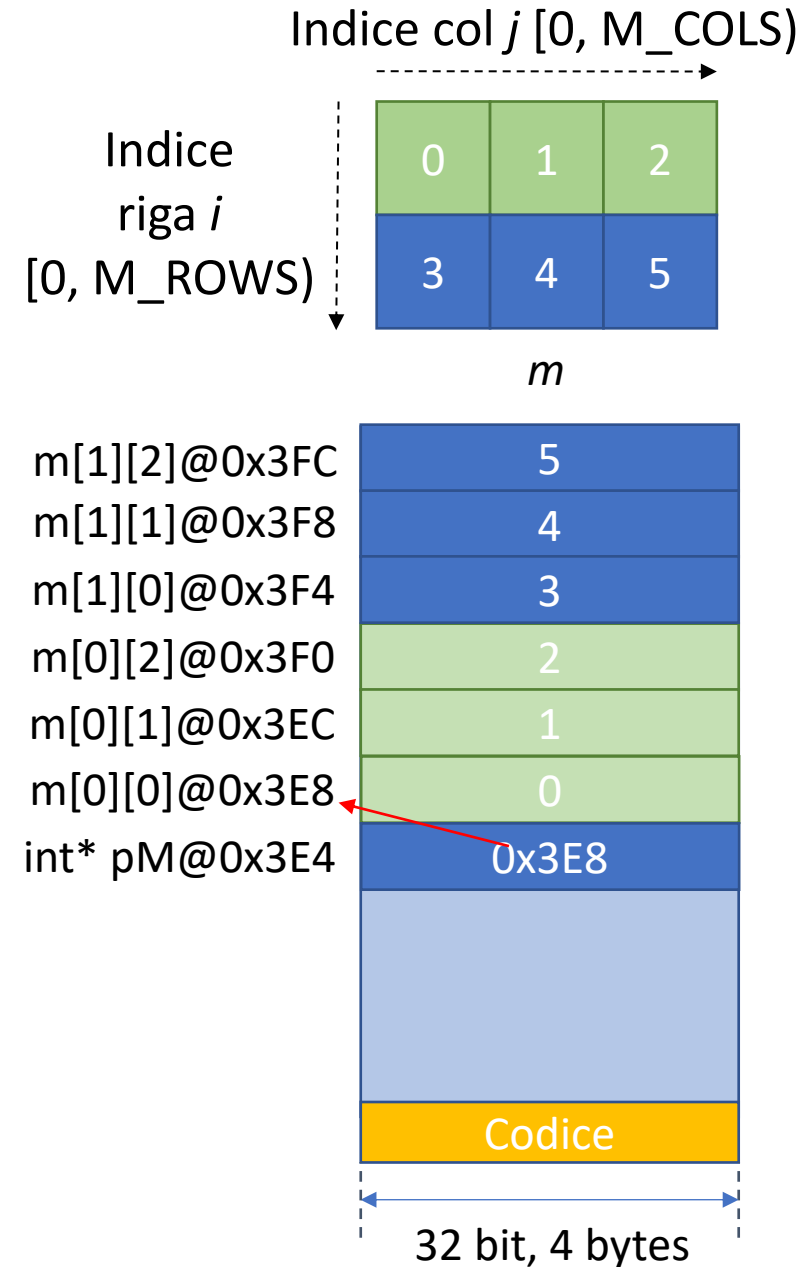
Casting
da `int*[2]`
a `int*`



Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    → {3,4,5}};
    int* pM = (int*) m;
    for (int i = 0; i < M_ROWS; i++) {
        for (int j = 0; j < M_COLS; j++) {
            printf ("m[%d][%d]@%p vale %d\n", i,
                j, pM, *pM);
            pM++;
        }
    }
}
```

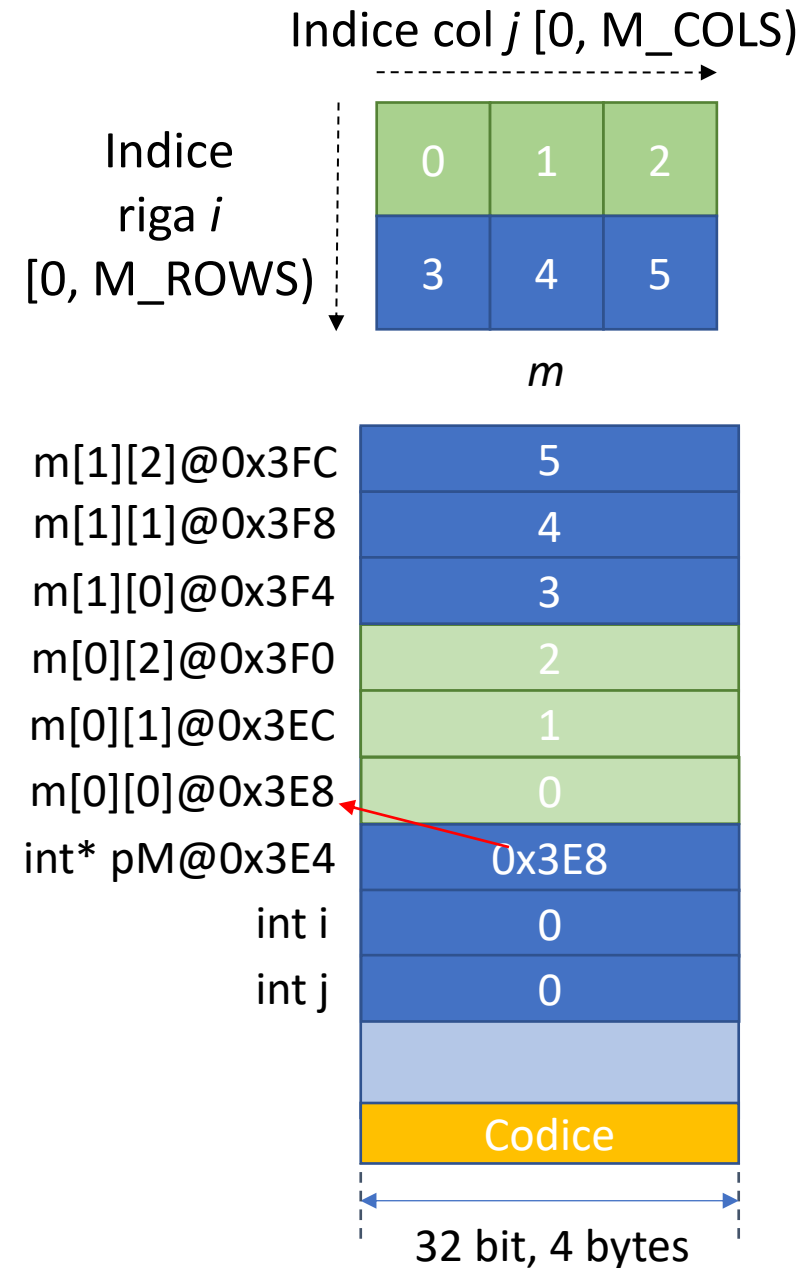


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pM = (int*) m;
    for (int i = 0; i < M_ROWS; i++) {
        for (int j = 0; j < M_COLS; j++) {
            printf ("m[%d][%d]@%p vale %d\n", i,
j, pM, *pM);
            pM++;
        }
    }
}
```

m[0][0]@0x3E8 vale 0

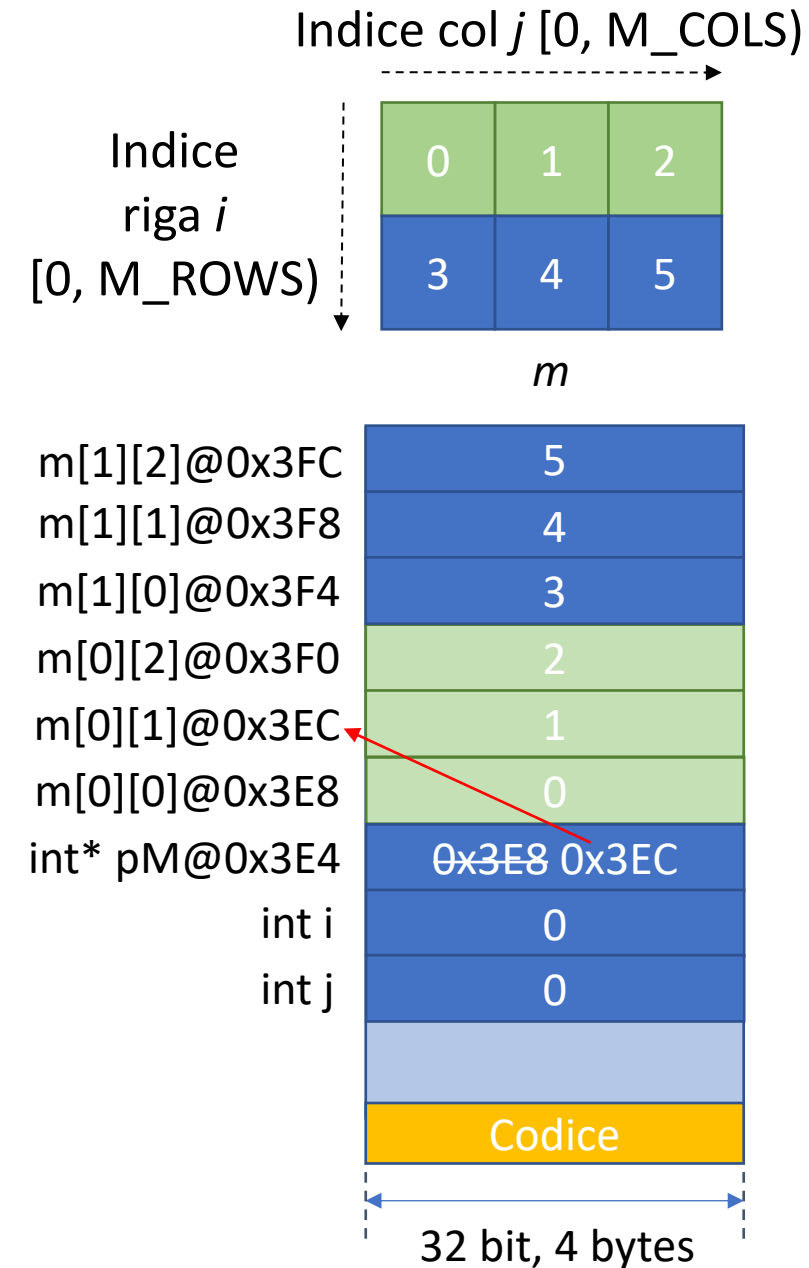


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pM = (int*) m;
    for (int i = 0; i < M_ROWS; i++) {
        for (int j = 0; j < M_COLS; j++) {
            printf ("m[%d][%d]@%p vale %d\n", i,
j, pM, *pM);
            pM++;
        }
    }
}
```

m[0][0]@0x3E8 vale 0

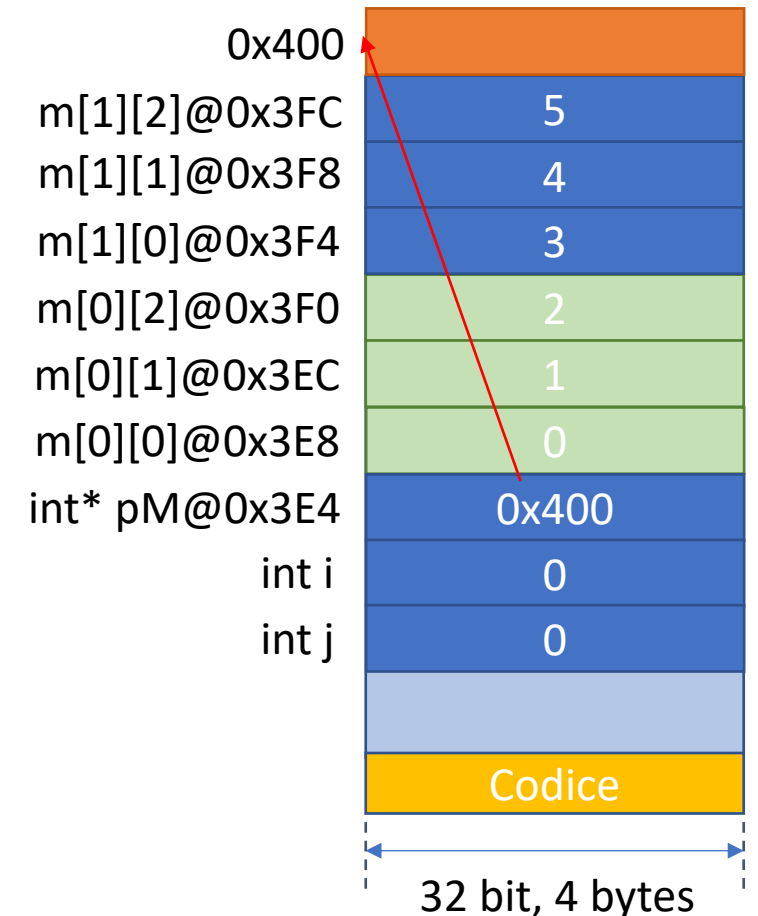


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pM = (int*) m;
    for (int i = 0; i < M_ROWS; i++) {
        for (int j = 0; j < M_COLS; j++) {
            printf ("m[%d][%d]@%p vale %d\n", i,
j, pM, *pM);
            pM++;
        } //end for su j
    } // end for su i
}
```

```
m[0][0]@0x3E8 vale 0
m[0][1]@0x3EC vale 1
m[0][2]@0x3F0 vale 2
m[1][0]@0x3F4 vale 3
m[1][1]@0x3F8 vale 4
m[1][2]@0x3FC vale 5
```

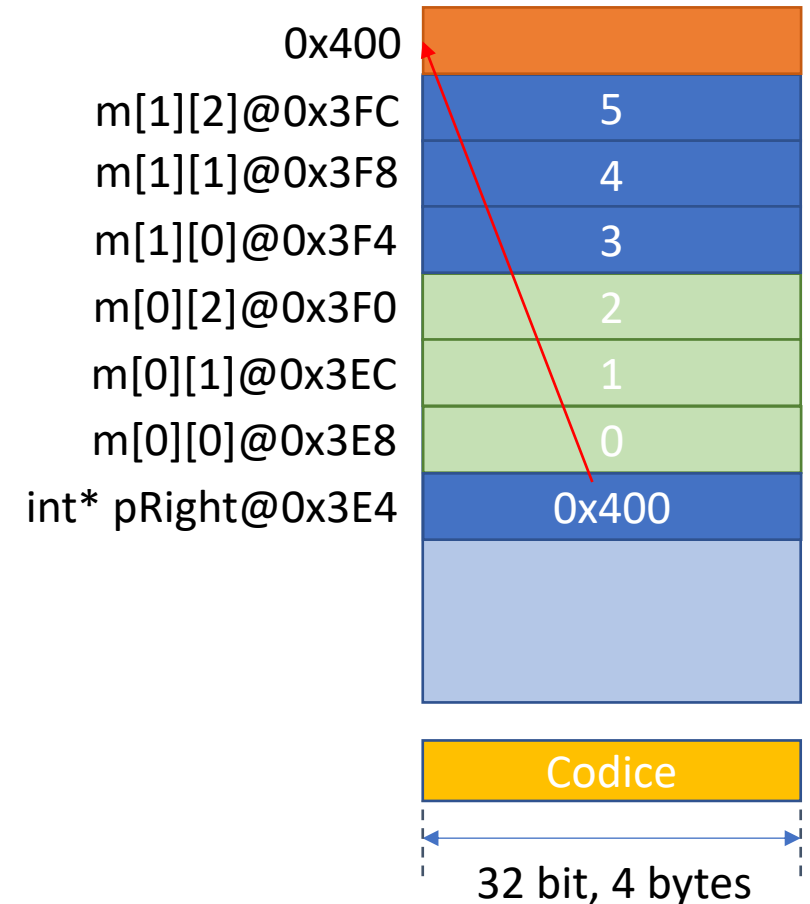


Puntatori a matrici

```
#define M_ROWS 2  
#define M_COLS 3
```

```
int main(void) {  
    int m[M_ROWS][M_COLS] = {{0,1,2},  
    {3,4,5}};  
    ➔ int* pRight = (int*) m + (M_ROWS *  
    M_COLS) * sizeof(int);  
    for (int* pM = (int*) m; pM < pRight;  
    pM++) {  
        printf ("int@%p vale %d\n", pM, *pM);  
    }  
}
```

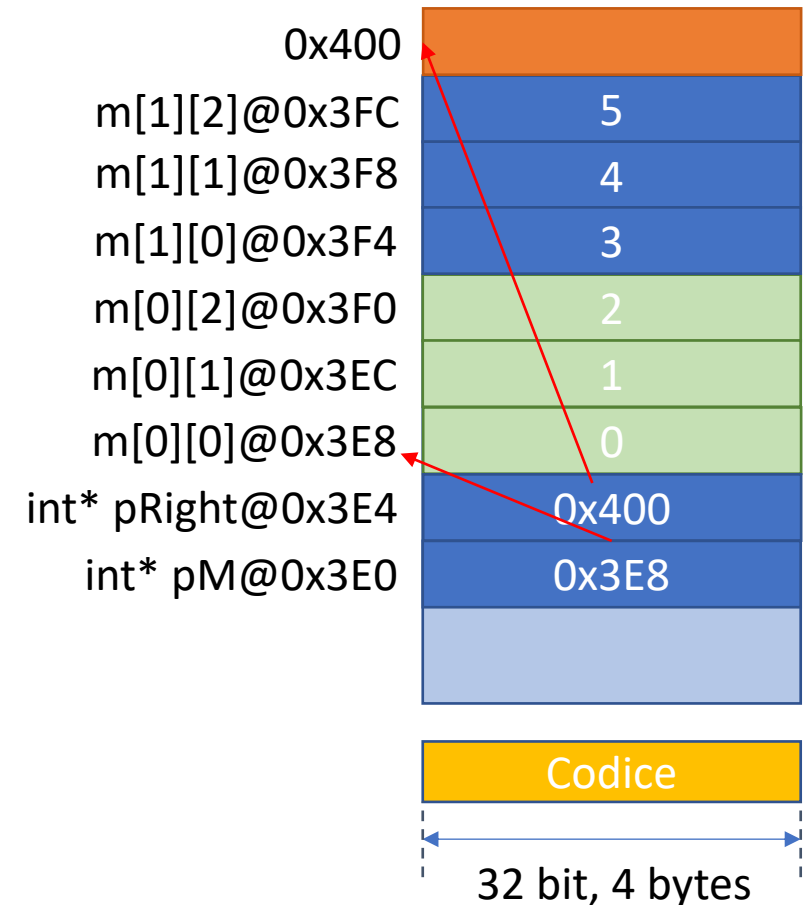
Vale l'aritmetica
dei puntatori!



Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

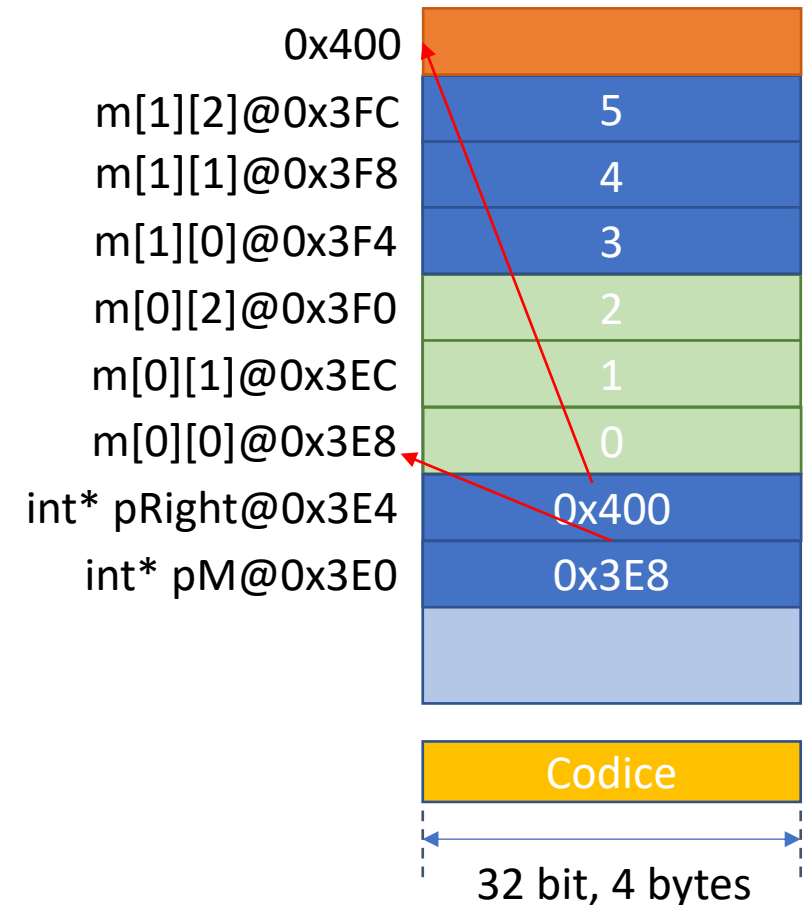
int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    ➔ for (int* pM = (int*) m; pM < pRight;
pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
}
```



Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    ➔ for (int* pM = (int*) m; pM < pRight;
pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
}
```

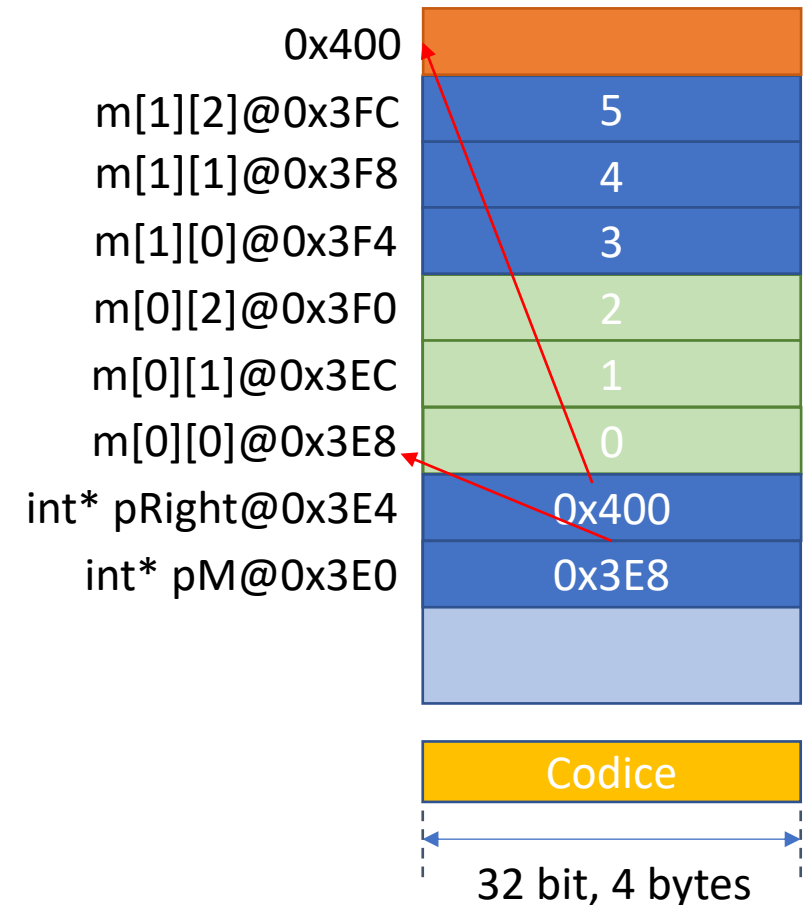


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    for (int* pM = (int*) m; pM < pRight;
pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
}
```

m[0][0]@0x3E8 vale 0

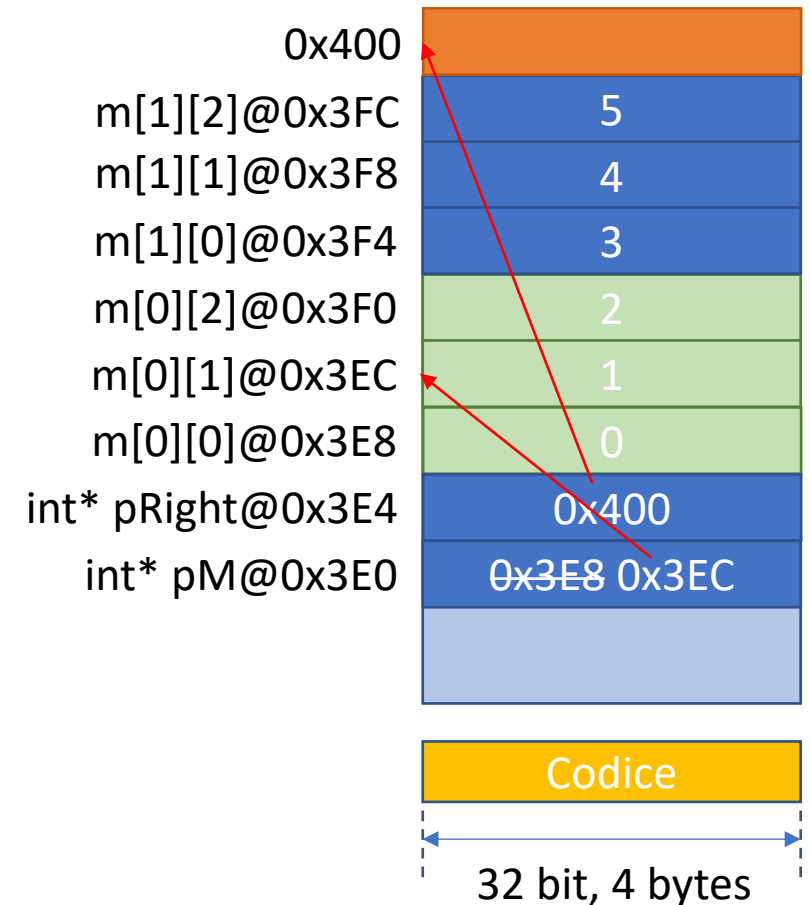


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    for (int* pM = (int*) m; pM < pRight;
➔ pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
}
```

m[0][0]@0x3E8 vale 0

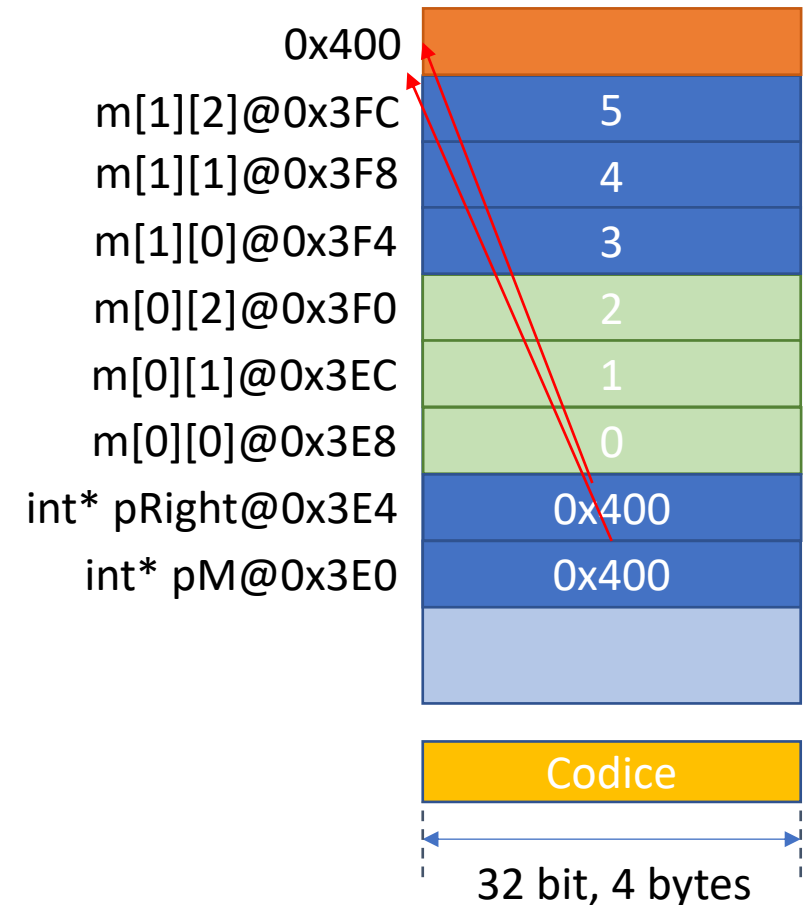


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    for (int* pM = (int*) m; pM < pRight;
pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
}
```

m[0][0]@0x3E8 vale 0
m[0][1]@0x3EC vale 1
m[0][2]@0x3F0 vale 2
m[1][0]@0x3F4 vale 3
m[1][1]@0x3F8 vale 4
m[1][2]@0x3FC vale 5

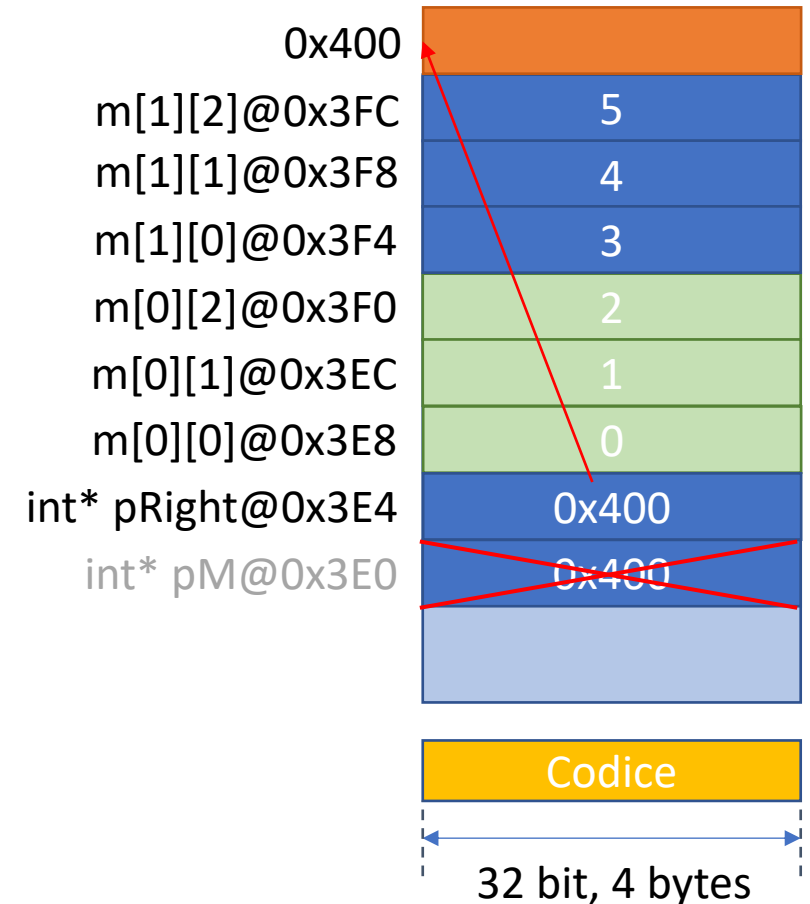


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    for (int* pM = (int*) m; pM < pRight;
pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
} //qui
```

m[0][0]@0x3E8 vale 0
m[0][1]@0x3EC vale 1
m[0][2]@0x3F0 vale 2
m[1][0]@0x3F4 vale 3
m[1][1]@0x3F8 vale 4
m[1][2]@0x3FC vale 5



Chiamata a funzione con matrici (ancora!?)

Signature *classica*: ho una costante per le colonne

```
#define STUDENTS 3
#define EXAMS 4

int minimum(const int grades[][EXAMS], size_t pupils, size_t tests);
int maximum(const int grades[][EXAMS], size_t pupils, size_t tests);
double average(const int setOfGrades[], size_t tests);
void printArray(const int grades[][EXAMS], size_t pupils, size_t tests);
```

Chiamata a funzione con matrici (ancora!?)

Signature *moderna*: ho delle variabili per le colonne (e le righe)

```
bool OgniRigaEsisteColonnaPari(int r, int c, int m[r][c]);
```

In questo caso sto implicitamente definendo un VLA: funziona solo su compilatori *moderni* (C99)

Funziona sia:

- su array-argomenti VLA
- su array-argomenti costanti

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Matrici con Stringhe

```
char fruit[3][7]= {"Apple", "Pear", "Orange"};
```

A	p	p	l	e	\0	\0
---	---	---	---	---	----	----

P	e	a	r	\0	\0	\0
---	---	---	---	----	----	----

O	r	a	n	g	e	\0
---	---	---	---	---	---	----

Matrici Sfrangiate

- Ragged Matrix, Irregular Matrix, Jagged Matrix: *a matrix having irregular numbers of items in each row*

		Columns			
		0	1	2	3
Rows	0	0	1	2	3
	1	4	5		
	2	6	7	8	

- Il C **NON** supporta nativamente le matrici sfrangiate
- Come simularle in maniera *ideomatica* in C?

Matrici Sfrangiate

- Usiamo un ulteriore array per memorizzare lo spazio occupato da ogni riga.

		Columns			
		0	1	2	3
Rows	0	0	1	2	3
	1	4	5		
	2	6	7	8	

```
D1=3; D2=4;3
```

```
int MatrixA[D1][D2] = {{0,1,2,3},{4,5},{6,7,8}};
```

```
int ArrayA[D1]={4,2,3};
```


Matrici ragged: Esercizio

- Un insegnamento magistrale è seguito nell' AA 22/23 da 3 studenti
- Ogni studente studente ha a disposizione fino a 4 appelli
 - Il primo studente ha conseguito 18 , 15 , 18 , 26
 - Il secondo studente ha conseguito 15 , 29
 - Il terzo studente ha conseguito 15 , 16 , 23
- Progettare la struttura dati necessaria a memorizzare i voti degli studenti dell'insegnamento tenendo conto del limite dei 4 appelli

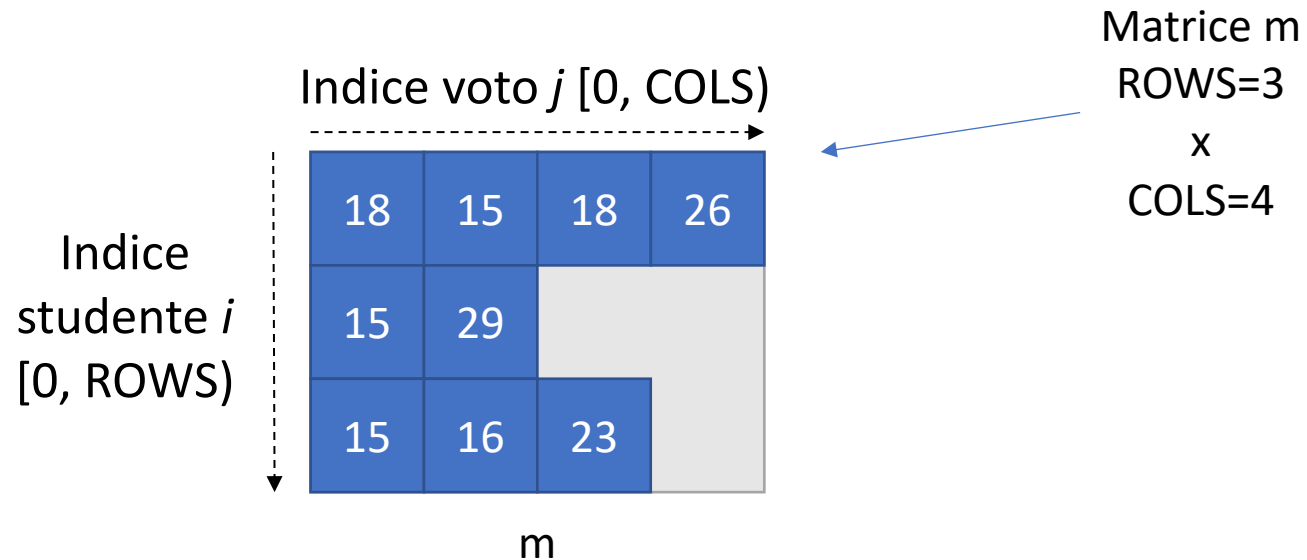
Matrici ragged

- Una possibilità è l'uso di matrici irregolari (*ragged*) dove il numero di righe è noto ma ogni riga ha lunghezza (numero colonne) diversa
- Per esempio, i voti dell'insegnamento saranno rappresentabili come

	1 voto	2 voto	3 voto	4 voto
Primo studente	18	15	18	26
Secondo studente	15	29		
Terzo studente	15	16	23	

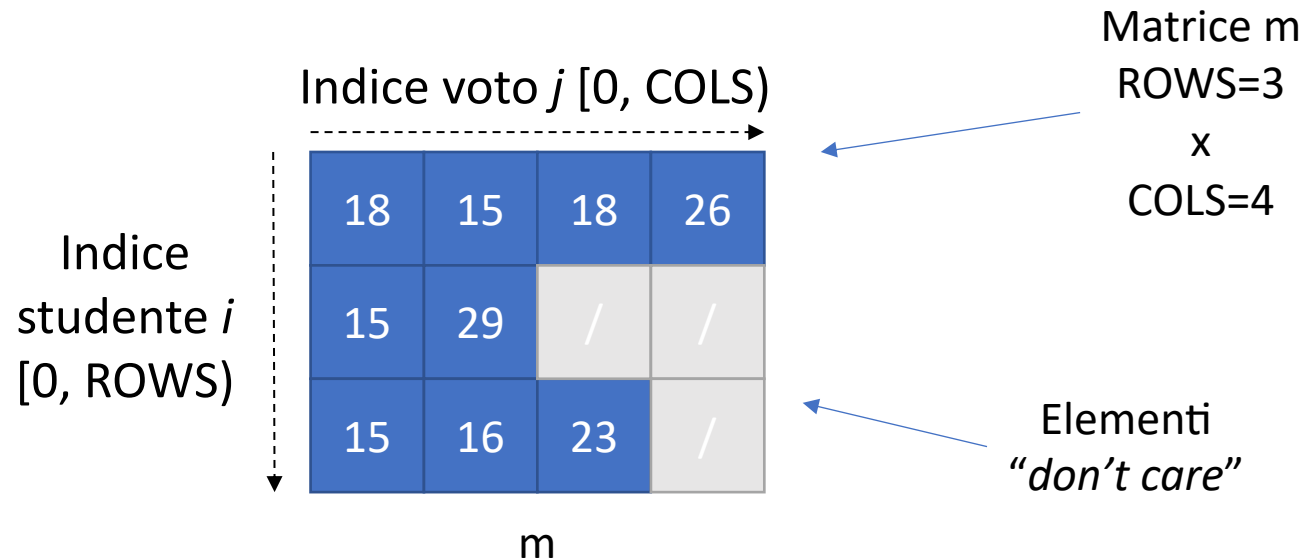
Matrici ragged

- Viene *preallocata* una matrice di $M_ROWS \times M_COLS$ dove M_COLS colonne sono sufficienti per la riga più lunga (4 appelli nell'esempio)



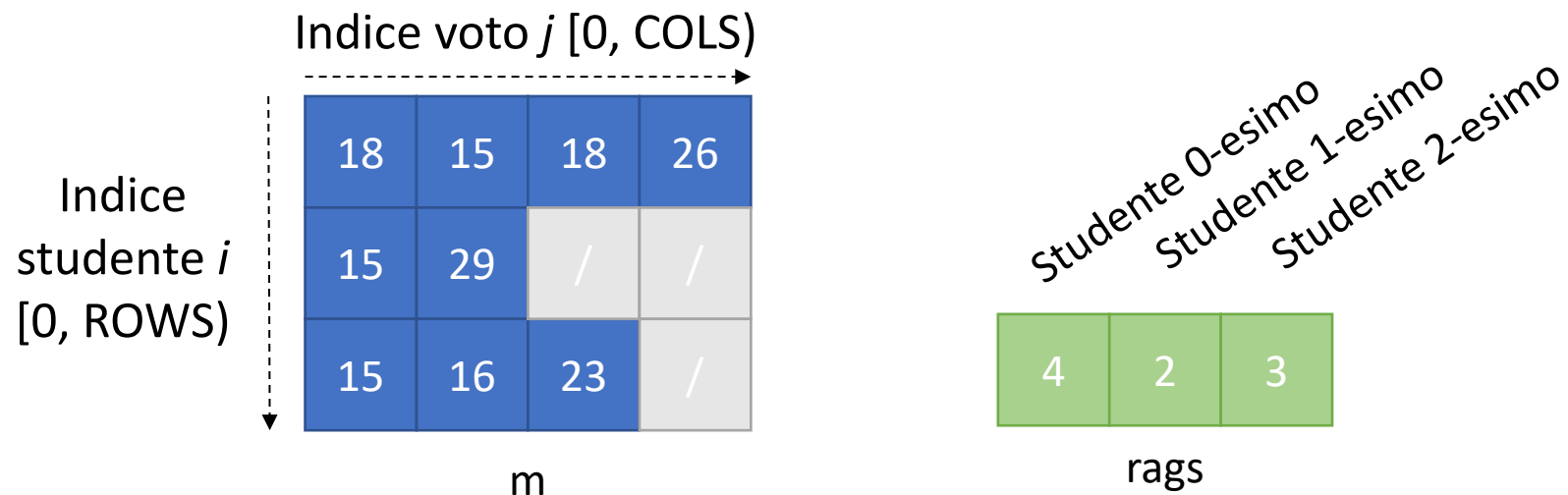
Matrici ragged

- Viene *preallocata* una matrice di $M_ROWS \times M_COLS$ dove M_COLS colonne sono sufficienti per la riga più lunga (4 appelli nell'esempio)
- Alcuni elementi della matrice sono da non considerare (*don't care*, /)



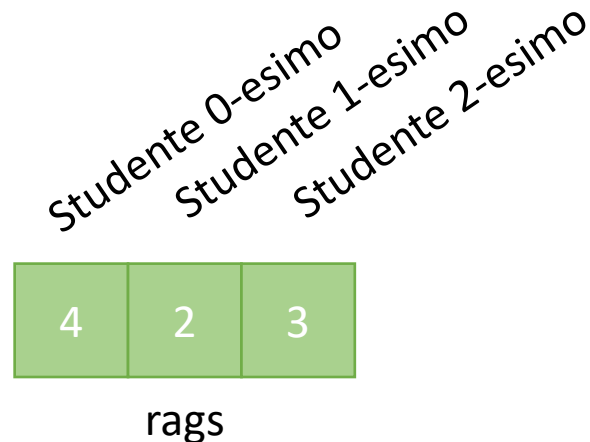
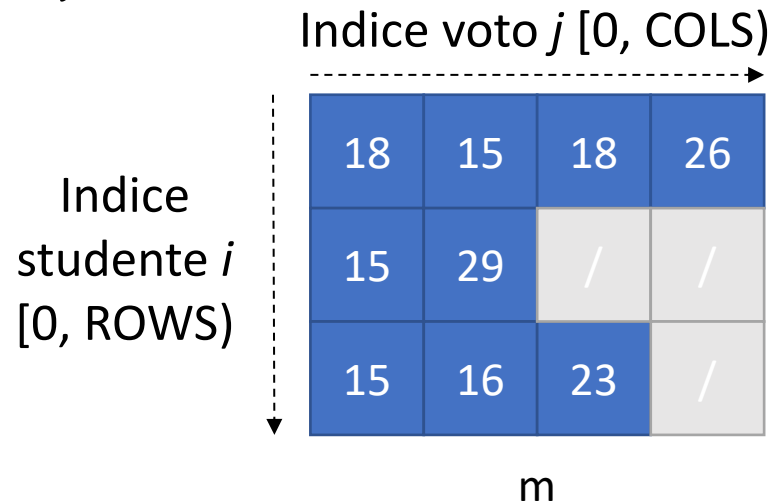
Matrici ragged

- Un vettore `rags[ROWS]` conta le colonne valide per ogni riga
- Per ogni riga i -esima, l'indice j è valido in $[0, \text{rags}[i])$
- Per ogni i -esima riga, vale $\text{rags}[i] \leq \text{COLS}$



Matrici ragged

```
#define ROWS 3
#define COLS 4
int main(void) {
    int m[ROWS][COLS]={ {18,15,18,26},
                          {15,29},
                          {15,16,23}};
    int rags[ROWS] = {4, 2, 3};
    stampa(ROWS, COLS, m, rags);
}
```



m[2][3]@0x3FC	?
m[2][2]@0x3F8	23
m[2][1]@0x3F4	16
m[2][0]@0x3F0	15
m[1][3]@0x3EC	?
m[1][2]@0x3E8	?
m[1][1]@0x3E4	29
m[1][0]@0x3E0	15
m[0][3]@0x3DC	26
m[0][2]@0x3D8	18
m[0][1]@0x3D4	15
m[0][0]@0x3D0	18
r[2]@0x3CC	3
r[1]@0x3C8	2
r[0]@0x3C4	4
Codice	

32 bit, 4 bytes

Matrici ragged

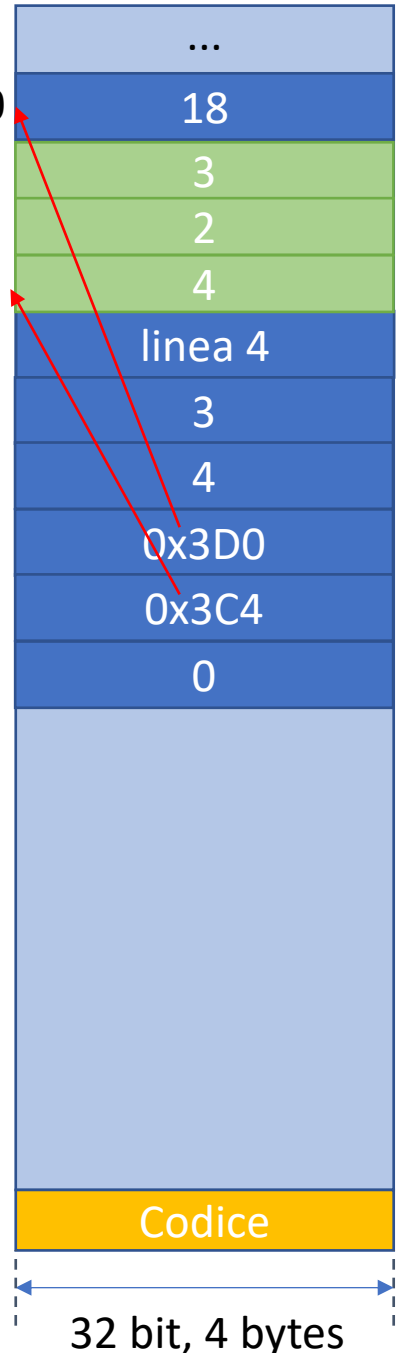
```
void stampa(size_t rows, size_t cols,  
            int m[rows][cols],  
            int rags[]) {  
    for (size_t i = 0; i < rows; i++) {  
        assert (rags[i] <= cols);  
        for (size_t j = 0; j < rags[i]; j++) {  
            printf("%2d ", m[i][j]);  
        }  
        printf("\n");  
    }  
}
```

Indice
studente i
[0, ROWS)

Indice voto j [0, COLS)			
18	15	18	26
15	29	/	/
15	16	23	/

m

$m[0][0]$ @0x3D0
 $r[2]$ @0x3CC
 $r[1]$ @0x3C8
 $r[0]$ @0x3C4
retl
int rows
int cols
int $m[][]$
int $rags[]$
int i



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```

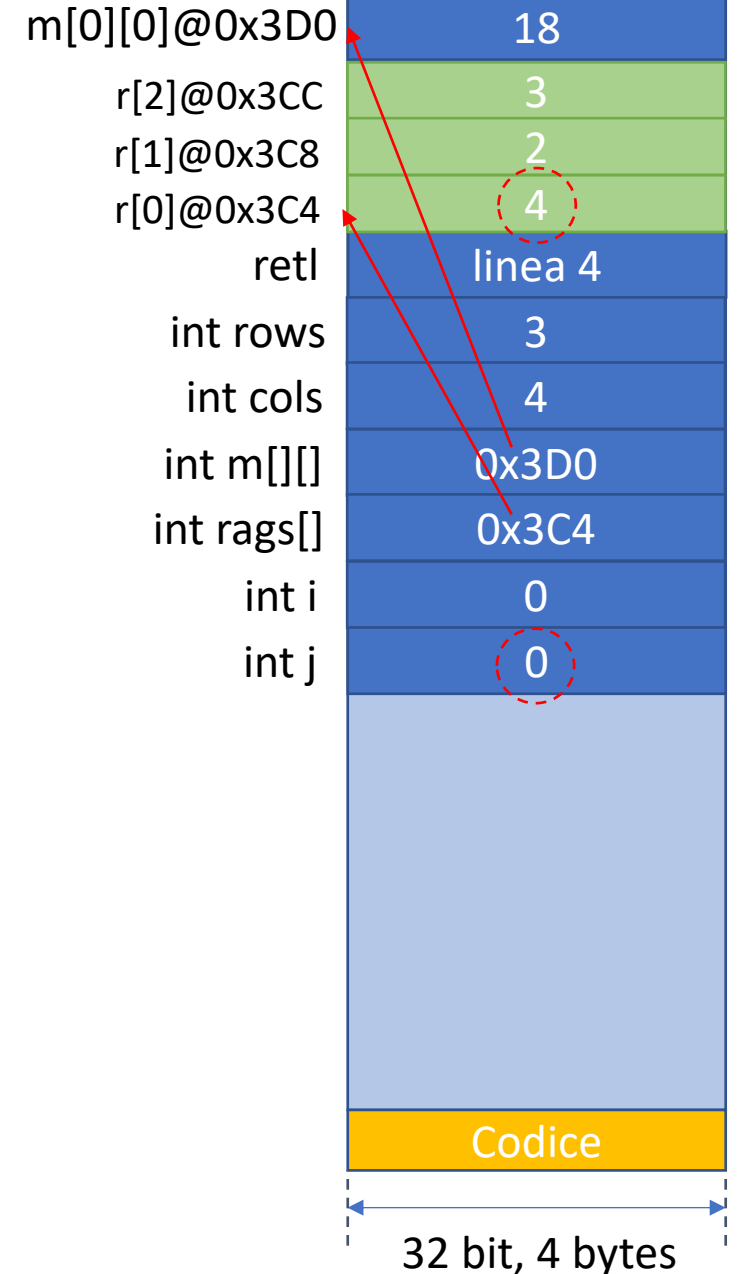


Indice voto j [0, COLS)

Indice studente i [0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```



Indice voto j [0, COLS)

Indice studente i [0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

18

m[0][0]@0x3D0

r[2]@0x3CC

r[1]@0x3C8

r[0]@0x3C4

retl

int rows

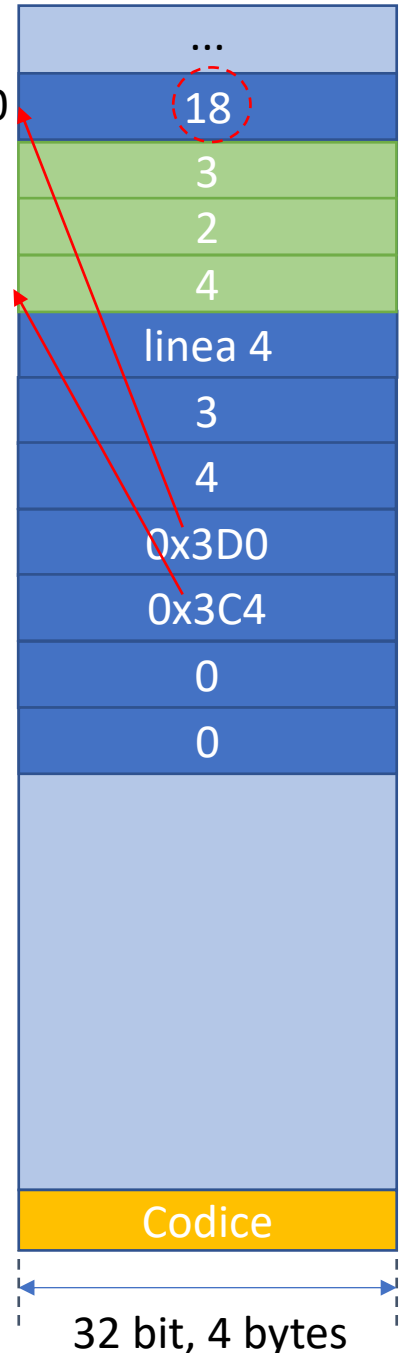
int cols

int m[][]

int rags[]

int i

int j



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```



Indice
studente i
[0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

18 15 18 26

$m[0][0]$ @0x3D0

$r[2]$ @0x3CC

$r[1]$ @0x3C8

$r[0]$ @0x3C4

retl

int rows

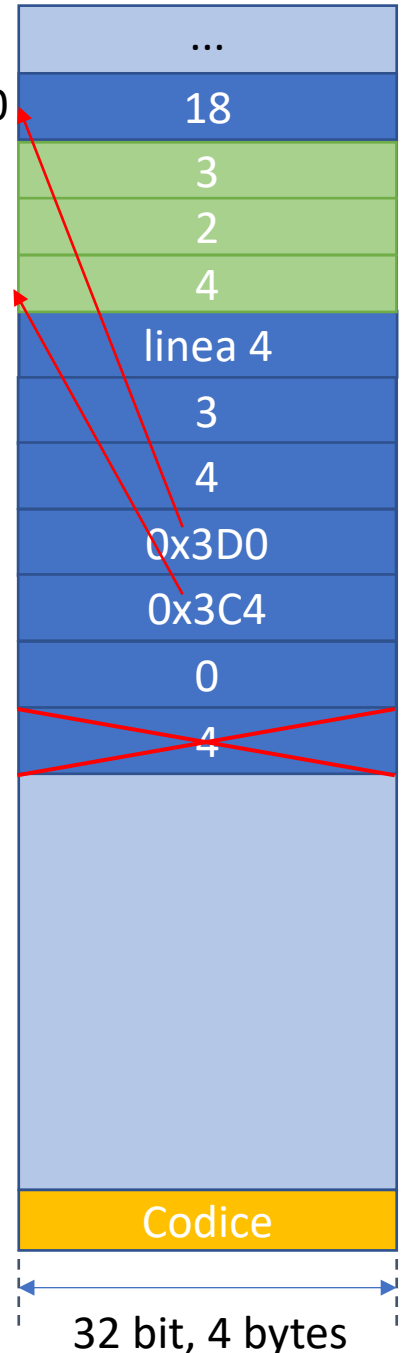
int cols

int $m[][]$

int rags[]

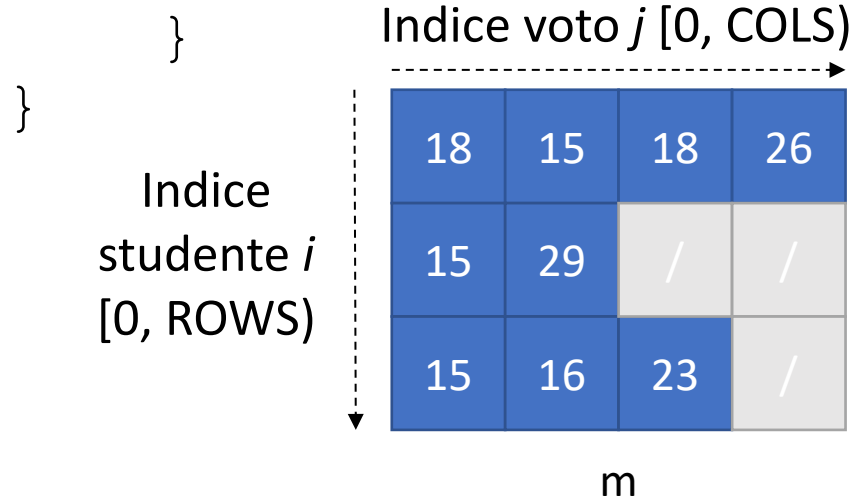
int i

int j



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
    }
}
```



18 15 18 26

m[0][0]@0x3D0

r[2]@0x3CC

r[1]@0x3C8

r[0]@0x3C4

retl

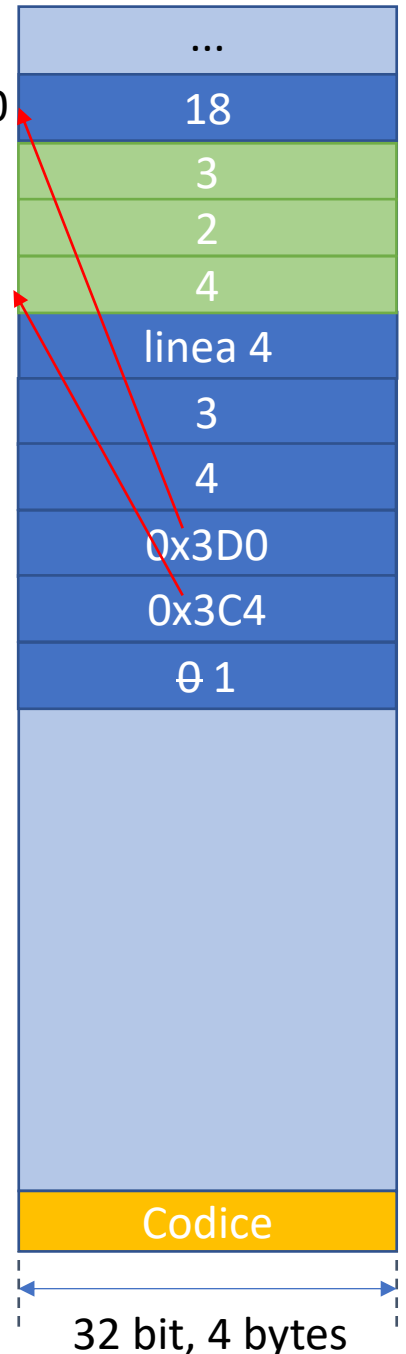
int rows

int cols

int m[][]

int rags[]

int i



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```

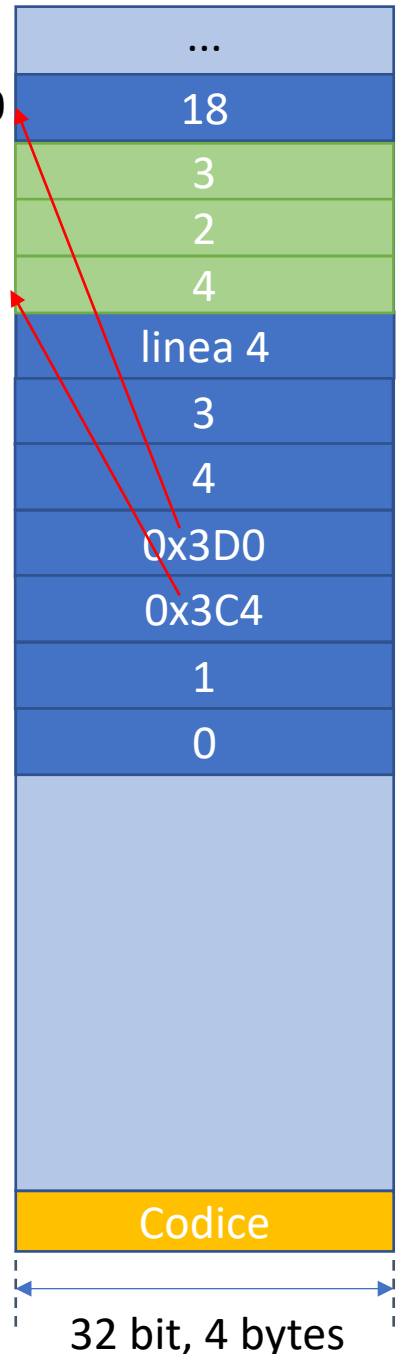
Indice
studente i
[0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

18 15 18 26
15

$m[0][0]$ @0x3D0
 $r[2]$ @0x3CC
 $r[1]$ @0x3C8
 $r[0]$ @0x3C4
retl
int rows
int cols
int $m[][]$
int $rags[]$
int i
int j



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```



Indice voto j [0, COLS)

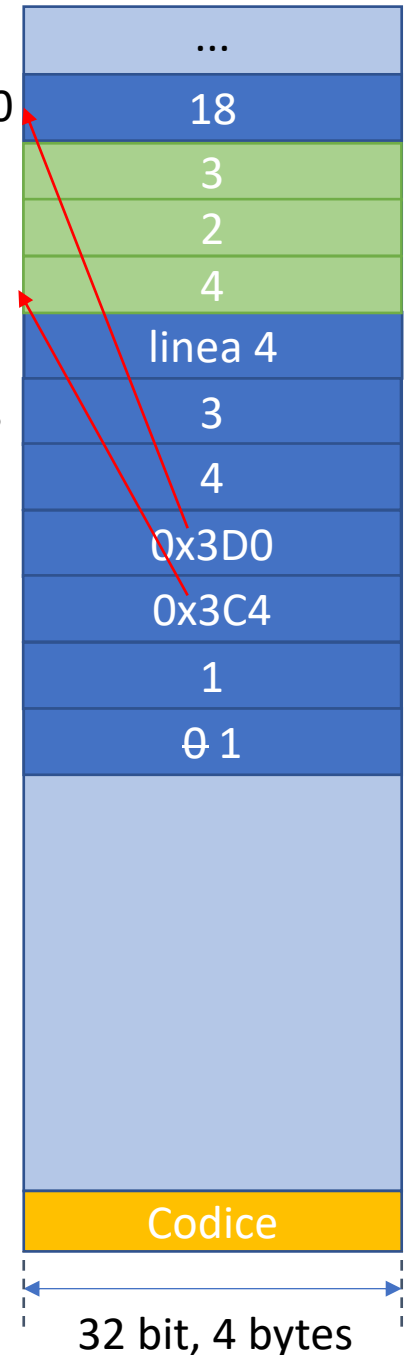
Indice studente i [0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

18 15 18 26
15

m[0][0]@0x3D0
r[2]@0x3CC
r[1]@0x3C8
r[0]@0x3C4
retl
int rows
int cols
int m[][]
int rags[]
int i
int j



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```



Indice voto j [0, COLS)

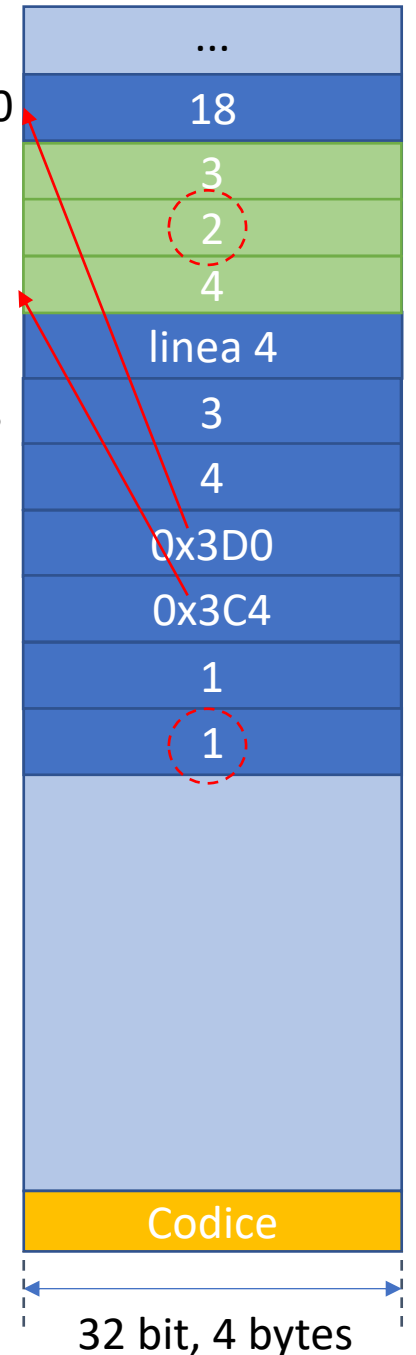
Indice studente i [0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

18 15 18 26
15

m[0][0]@0x3D0
r[2]@0x3CC
r[1]@0x3C8
r[0]@0x3C4
retl
int rows
int cols
int m[][]
int rags[]
int i
int j



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```



Indice voto j [0, COLS)

Indice studente i [0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

```
18 15 18 26
15 29
```

m[0][0]@0x3D0

r[2]@0x3CC

r[1]@0x3C8

r[0]@0x3C4

retl

int rows

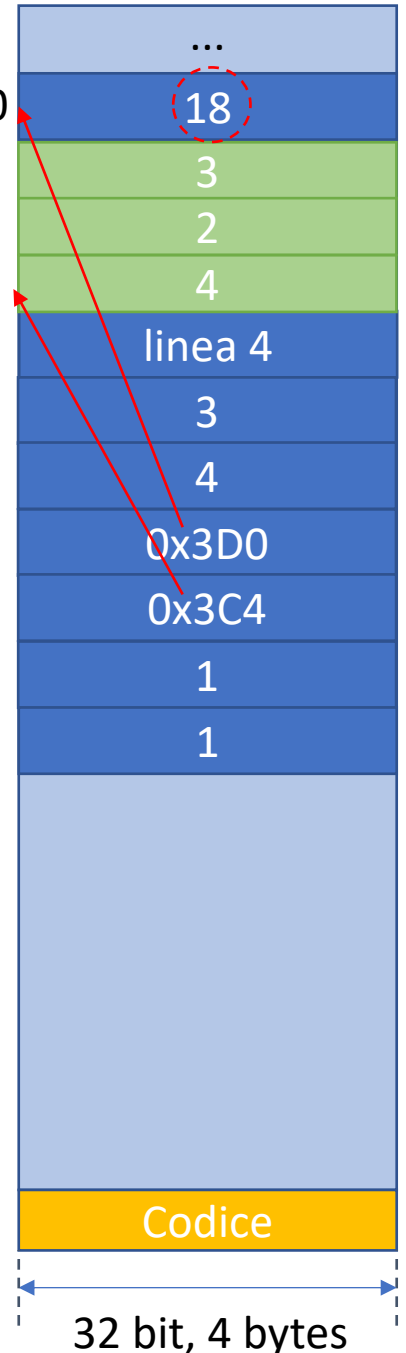
int cols

int m[][]

int rags[]

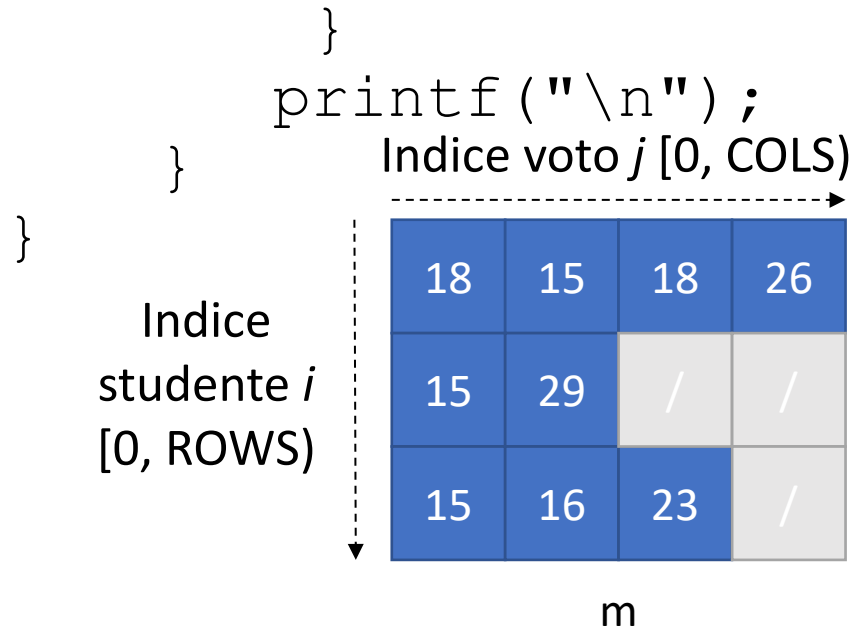
int i

int j



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
    }
}
```



```
18 15 18 26
15 29
```

`m[0][0]@0x3D0`

`r[2]@0x3CC`

`r[1]@0x3C8`

`r[0]@0x3C4`

`retl`

`int rows`

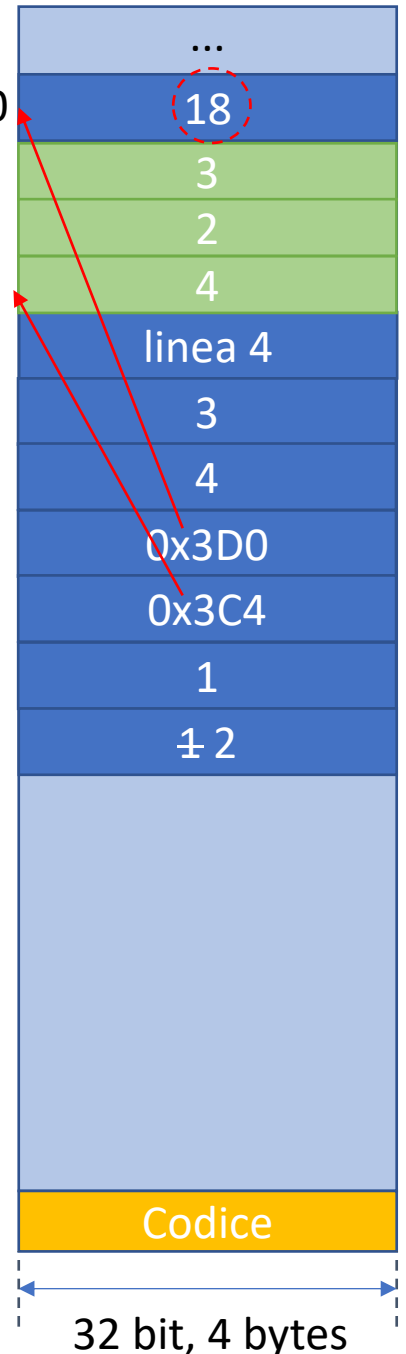
`int cols`

`int m[][]`

`int rags[]`

`int i`

`int j`



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```



Indice voto j [0, COLS)

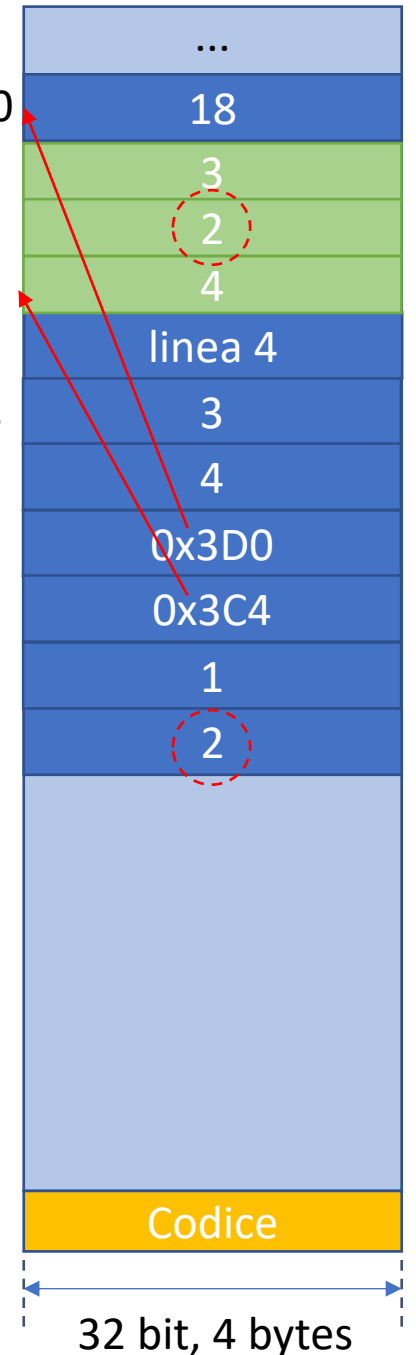
Indice studente i [0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

```
18 15 18 26
15 29
```

m[0][0]@0x3D0
r[2]@0x3CC
r[1]@0x3C8
r[0]@0x3C4
retl
int rows
int cols
int m[][]
int rags[]
int i
int j



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```



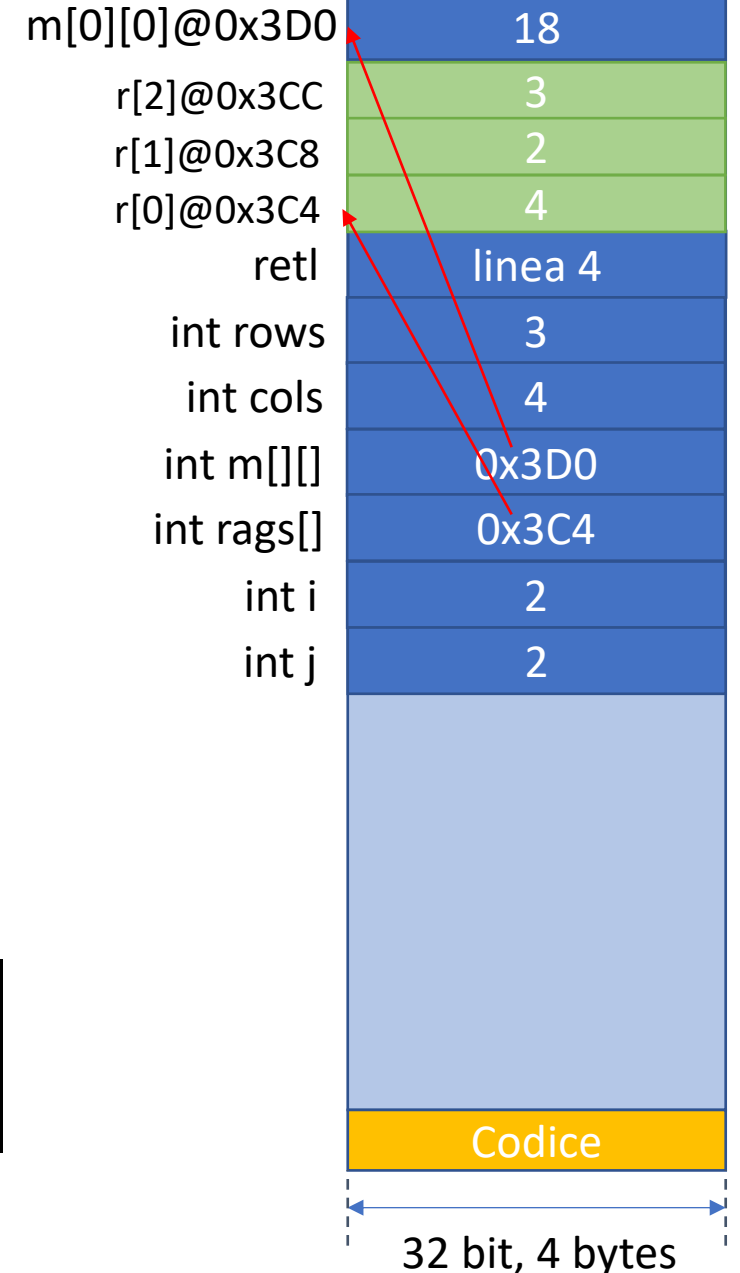
Indice voto j [0, COLS)

Indice studente i [0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

```
18 15 18 26
15 29
15 16 23
```



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```



Indice voto j [0, COLS)

Indice studente i [0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

```
18 15 18 26
15 29
15 16 23
```

m[0][0]@0x3D0

r[2]@0x3CC

r[1]@0x3C8

r[0]@0x3C4

retl

int rows

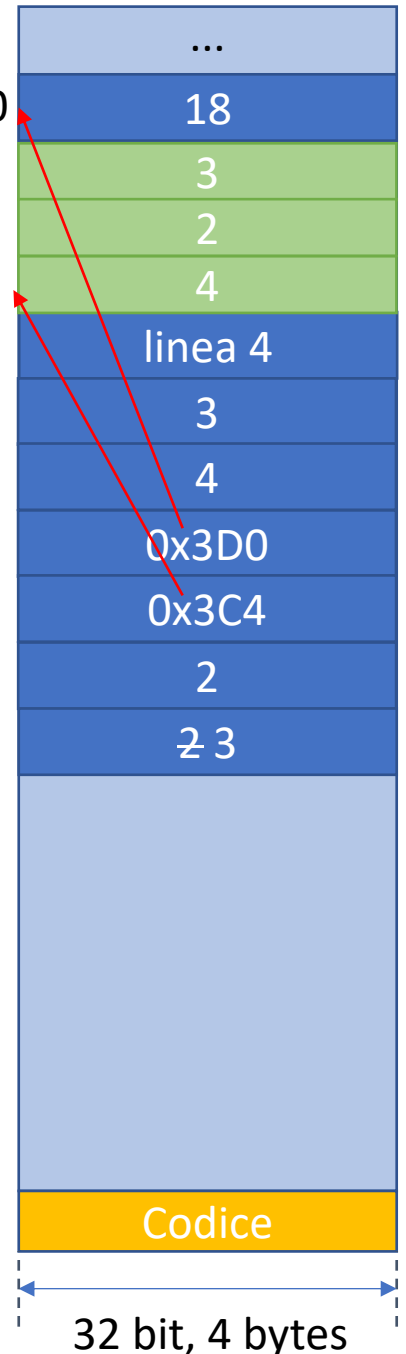
int cols

int m[][]

int rags[]

int i

int j



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```



Indice
studente i
[0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

Indice voto j [0, COLS)

```
18 15 18 26
15 29
15 16 23
```

m[0][0]@0x3D0

r[2]@0x3CC

r[1]@0x3C8

r[0]@0x3C4

retl

int rows

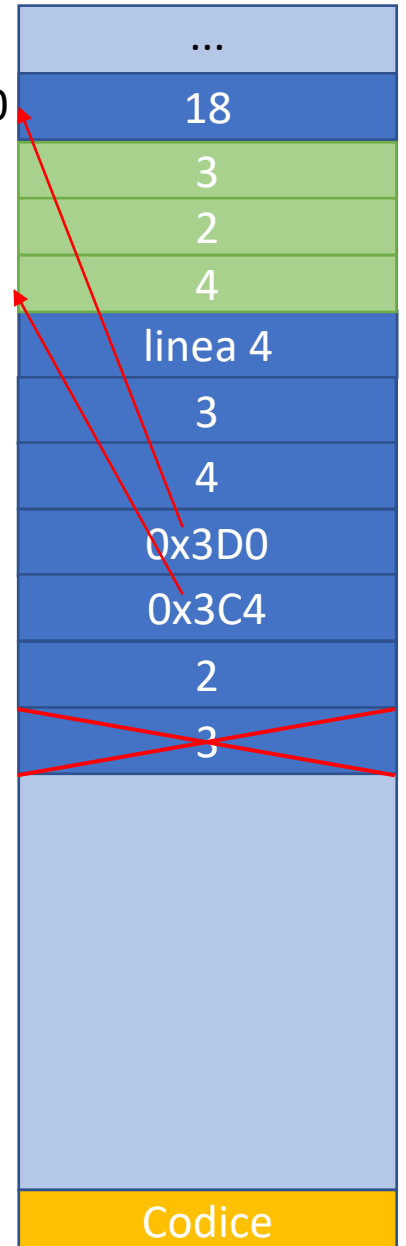
int cols

int m[][]

int rags[]

int i

int j



32 bit, 4 bytes

Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```



Indice
studente i
[0, ROWS)

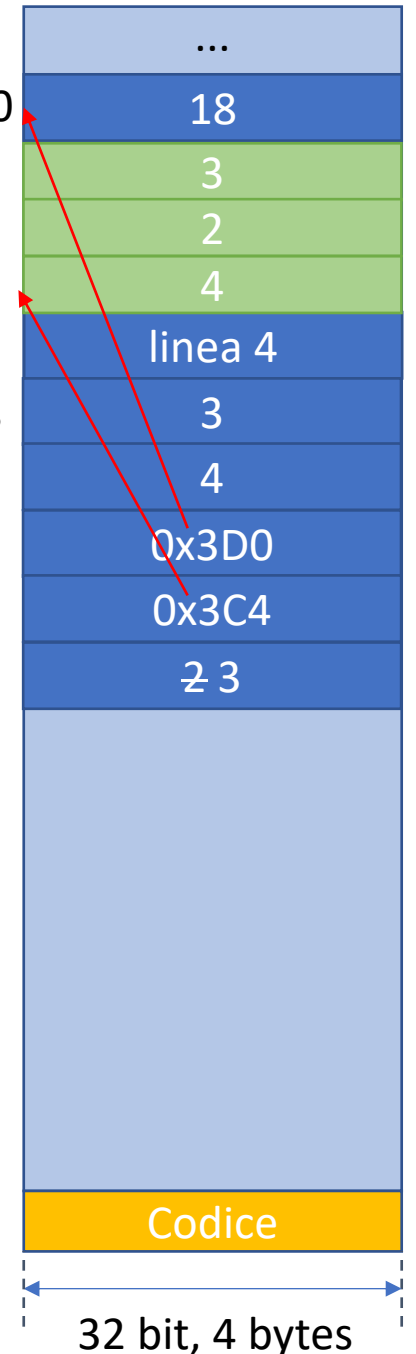
Indice voto j [0, COLS)

18	15	18	26
15	29	/	/
15	16	23	/

m

```
18 15 18 26
15 29
15 16 23
```

$m[0][0]$ @0x3D0
 $r[2]$ @0x3CC
 $r[1]$ @0x3C8
 $r[0]$ @0x3C4
 retl
 int rows
 int cols
 int $m[][]$
 int $rags[]$
 int i



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```

Indice voto j [0, COLS)

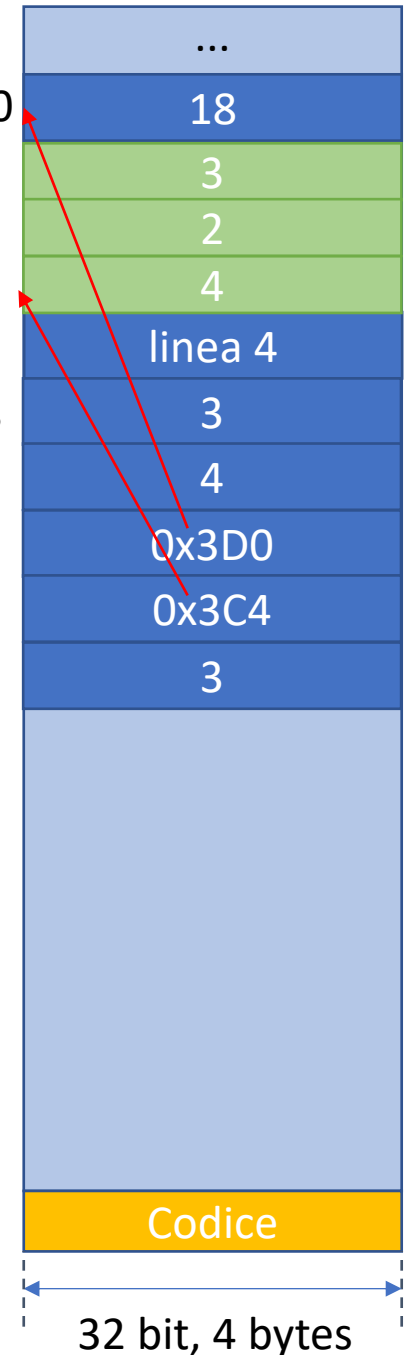
Indice studente i [0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

```
18 15 18 26
15 29
15 16 23
```

m[0][0]@0x3D0
r[2]@0x3CC
r[1]@0x3C8
r[0]@0x3C4
retl
int rows
int cols
int m[][]
int rags[]
int i



Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
        printf("\n");
    }
}
```

Indice voto j [0, COLS)

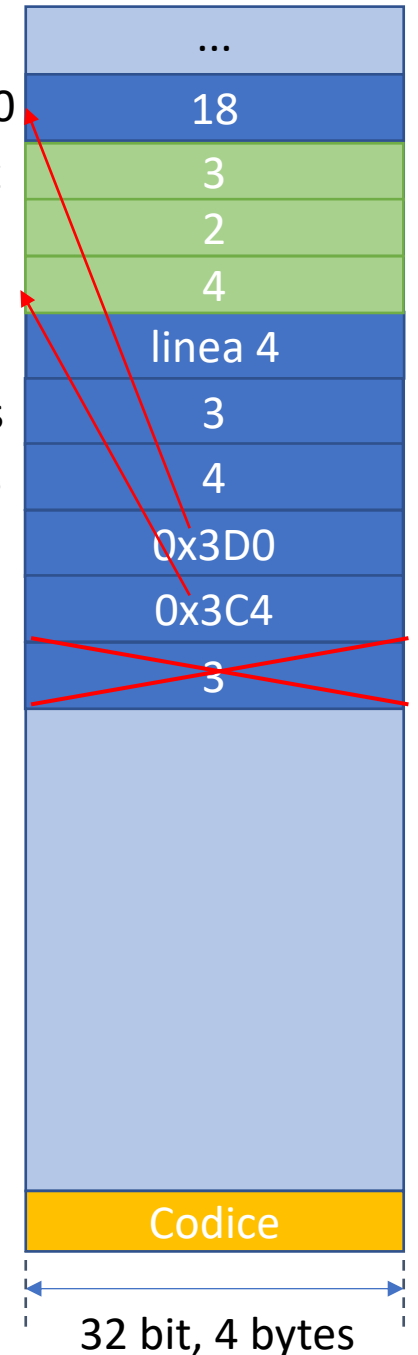
Indice studente i [0, ROWS)

18	15	18	26
15	29	/	/
15	16	23	/

m

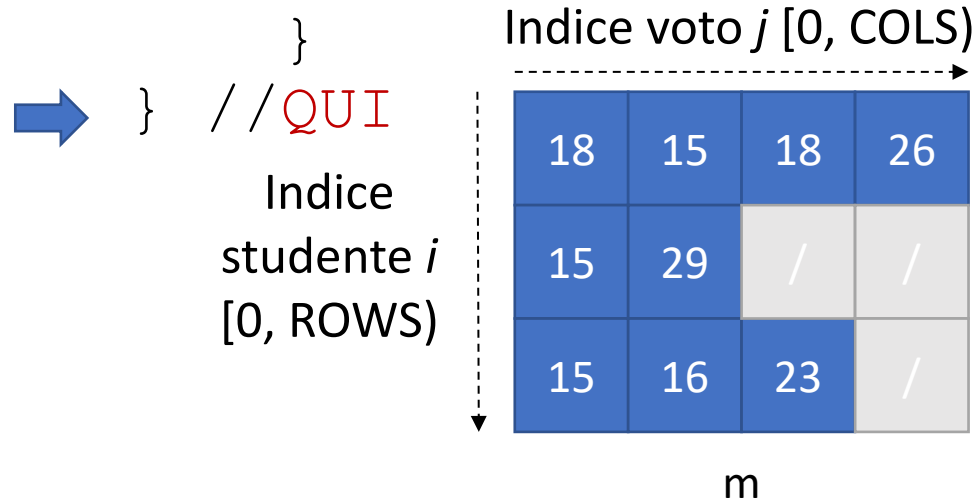
```
18 15 18 26
15 29
15 16 23
```

m[0][0]@0x3D0 18
 r[2]@0x3CC 3
 r[1]@0x3C8 2
 r[0]@0x3C4 4
 retl linea 4
 int rows 3
 int cols 4
 int m[][] 0x3D0
 int rags[] 0x3C4
 int i 3

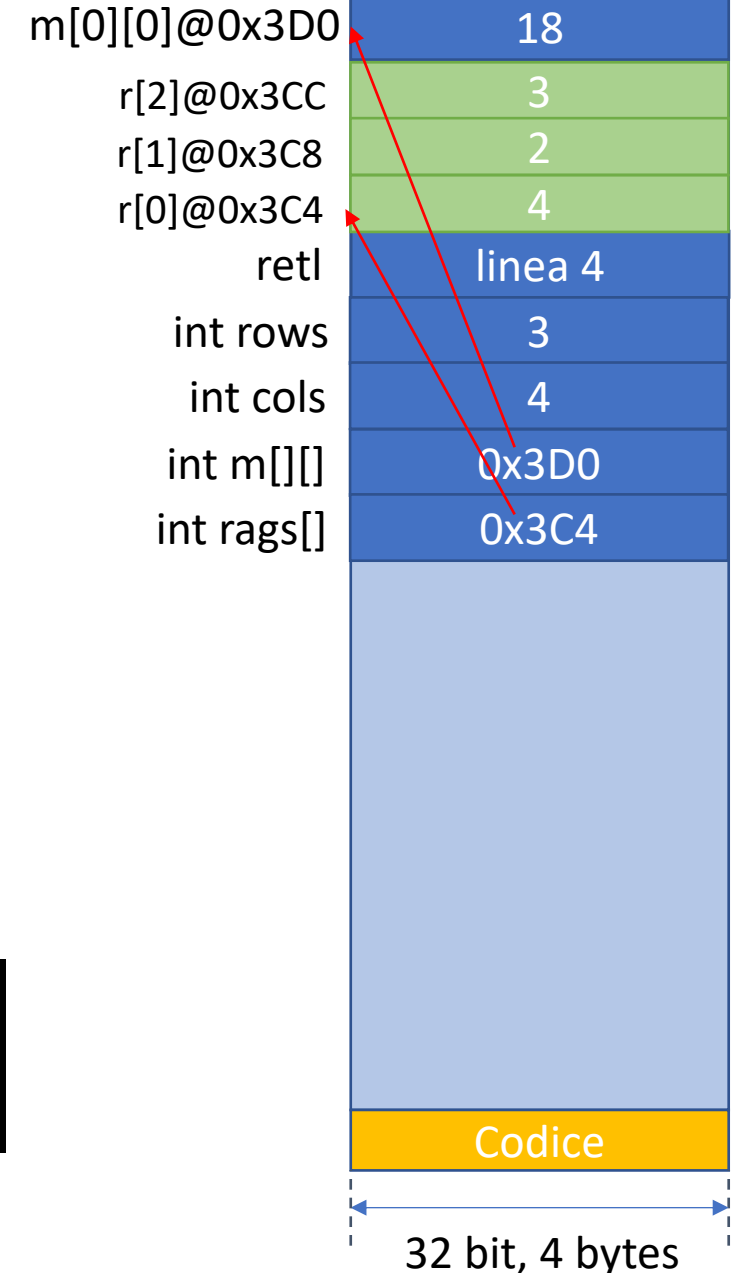


Matrici ragged

```
void stampa(size_t rows, size_t cols,
            int m[rows][cols],
            int rags[]) {
    for (size_t i = 0; i < rows; i++) {
        assert (rags[i] <= cols);
        for (size_t j = 0; j < rags[i]; j++) {
            printf("%2d ", m[i][j]);
        }
    }
}
```



```
18 15 18 26
15 29
15 16 23
```



Esercizio su Sfrangiate

- Scrivere una funzione che restituisca true se le due matrici sfrangiate in input sono uguali.

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Esercizi su matrici 1

- Scrivere una funzione che dati una matrice bidimensionale a , ed un valore di riferimento v , restituisce un array che contiene tutti e soli gli elementi di a che sono il doppio di v .
- Scrivere una funzione che data una matrice bidimensionale $a \neq \text{null}$, restituisce un array s in cui ogni elemento $s[i]$, contiene la somma degli elementi della i -esima riga in a .
- Scrivere una funzione che restituisce un array s in cui ogni l'elemento $s[j]$ contiene la somma degli elementi della j -esima colonna in a .

Esercizi su matrici 2

Sia data una matrice bidimensionale a .

- Dato anche un intero `limiteSuperiore`, scrivere una funzione C che restituisce un array con tutti e soli i valori interi in a minori del valore `limiteSuperiore`;
- Scrivere una funzione C che restituisce un array con tutti e soli gli elementi dispari di a ;
- Scrivere una funzione C che restituisce un array con tutti e soli gli elementi che in a occupano una riga di indice dispari avendo, simultaneamente, valore pari;
- Dati anche due valori `min` e `max`, scrivere una funzione C che restituisce tutti e soli gli elementi di a compresi tra `min` incluso e `max` escluso;
- Dati anche due numeri naturali m , n , non superiori, rispettivamente, al numero di righe e di colonne in a , scrivere una funzione C che restituisce una matrice $m \times n$ in cui ogni elemento all'intersezione della riga i e della colonna j contiene il valore $i+j$.

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Dangling Pointer

```
int* copy(int a[], int size);

int main(void) {
    int  nums[4]={3,4,5,6};
    int* nums2 = copy(nums,4); //dangling pointer HERE
    ...
}

int* copy(int a[], int size)
{
    int i;
    int a2[size];
    for(int i=0;i<size;i++){
        a2[i] = a[i];
    }
    return a2;
}
```

Longevità dei dati

Vorremmo che nei nostri programmi C ci fossero dati:

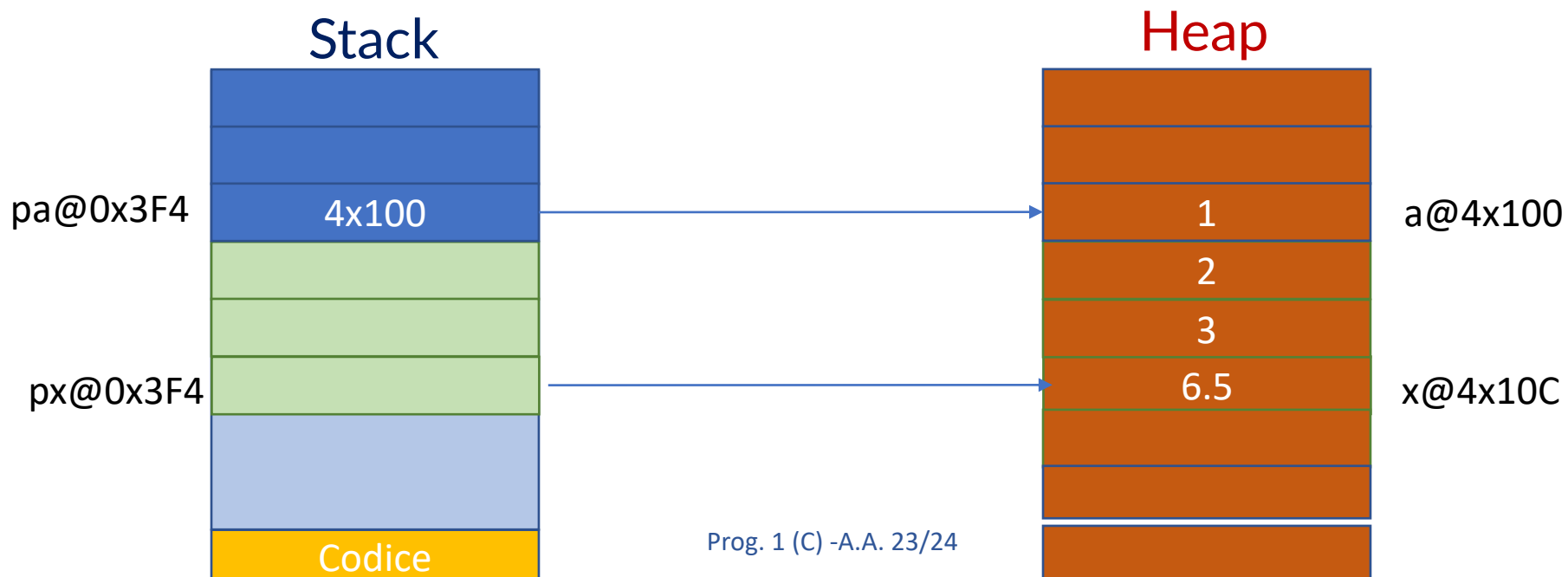
- **dinamici**: la dimensione dell'array cambia in base all'input dell'utente -> VLA
- **longevi/persistenti**: non scompaiono dopo la fine della funzione
- **più grandi**: lo stack è (relativamente) piccolo

La **soluzione canonica di Prog. 1** per la persistenza è dichiarare le variabili nella chiamante (tipicamente main) e passarli come "parametri formali di uscita"

P.S.: dichiarare le variabili globali (*il lato oscuro* ... NO!!!)

La HEAP

- heap: memoria che può essere utilizzato per allocare dati dinamici, persistenti e di grandi dimensioni
- Devo però rinunciare alla allocazione automatica: alloco e dealloco **manualmente**



malloc Manual ALLOCATION, `stdlib.h`

```
pointer_variable = (type*) malloc(size);
```

- La funzione `malloc` alloca un blocco di memoria heap di una data dimensione
 - restituisce un puntatore al primo byte di tale memoria.
 - è necessario eseguire il cast del puntatore restituito al tipo appropriato
 - inizialmente la memoria contiene dati spazzatura
 - spesso usato con `sizeof` per allocare memoria per un dato tipo di dati

```
// int a[8]; <-- stack
```

```
int* a = (int*) malloc(8 * sizeof(int));
```

```
a[0] = 10;
```

```
a[1] = 20;
```

```
...
```

Heap array

```
int* copy(int a[], int size);
int main(void) {
    int nums[4] = {7, 4, 3, 5};
    int* nums2 = copy(nums, 4);
    ...
    return 0;
}
int* copy(int a[], int size) { int i;
    int* a2 = (int*) malloc(size * sizeof(int));
    for (i = 0; i < size; i++) {
        a2[i] = a[i];
    }
    return a2;
}
```

NULL

NULL: una posizione di memoria non valida a cui non è possibile accedere.

- in C, NULL è una costante globale il cui valore è 0.
- se si effettua una malloc ma non c'è memoria libera, viene restituito NULL.
- è possibile inizializzare un puntatore a NULL se non ha un valore significativo.
- dereferenziare un puntatore nullo manda in crash il programma.

```
int* p = NULL;  
*p = 42; // segfault
```

- **Esercizio:** Scrivete un programma che calcoli quanto sono grandi lo stack e l'heap di un programma C predefinito.

Deallocare la memoria

- la memoria heap rimane occupata fino alla fine del programma
- **garbage collector**: un processo che recupera automaticamente la memoria che non è più in uso. Tiene traccia di quali variabili puntano a quale memoria, ecc. Utilizzato in Java e in molti altri linguaggi moderni, ma (purtroppo) **non in C**.

```
// Java
public static int[] f() {
    int[] a = new int[1000];
    int[] a2 = new int[1000]; return a2;
}
```

Memory leaks

- perdita di memoria: mancato rilascio della memoria quando non è più necessaria.
- Capità spesso in C
- Può essere un problema se il programma viene eseguito per molto tempo (quando il programma esce, tutta la sua memoria viene restituita al sistema operativo).

```
void f(void) {  
    int* a = (int*) malloc(1000*sizeof(int));  
    ...  
}
```

free

- rilascia la memoria puntata dal puntatore dato

precondizione: il puntatore deve fare riferimento a un blocco di memoria allocato nell'heap che non sia già stato liberato

```
int* a = (int*) malloc(8*sizeof(int)) ;
```

```
...
```

```
free(a) ;
```

- è considerata una buona pratica impostare in puntatore a NULL dopo averlo liberato:

```
free(a) ;
```

```
a=NULL;
```

Corrompere la memoria

- se il puntatore passato a free non punta a un blocco allocato nell'heap, o se il blocco è già stato liberato, succedono cose brutte

```
int* a1 = (int*) malloc(1000 * sizeof(int));  
int a2[1000];  
int* a3;  
int* a4 = NULL;  
free(a1); //OK  
free(a1); //Già free  
free(a2); //No heap  
free(a3); //No heap  
free(a4); //No heap
```

- si è fortunati se si blocca, piuttosto che *corrompere* silenziosamente qualcosa
(*memory corruption*)

Summary: 3 modi per creare un array

- Automatic

```
int arrayAutomatic[100];
```

- Manual

```
int* arrayManual=(int*) malloc(100*sizeof(int)); // PROG2!
```

- VLA (C99)

```
int size;
```

```
scanf("%d",&size);
```

```
int arrayVLA[size];
```


Manual vs. VLA

```
int* vlamal() {  
    int n;  
    int* pi;  
    scanf("%d", &n);  
  
    pi = (int *) malloc (n * sizeof(int));  
    int ar[n]; // vla  
  
    pi[2] = ar[2] = -5;  
    return pi; // ar??  
}
```

Esercizio malloc

Scrivere una funzione che legge una sequenza di interi e la stampi al contrario, allocando la memoria con `malloc`. Due versioni:

1. L'input è dato da un intero `n` e da una sequenza di `n` numeri; basta quindi una sola chiamata di `malloc` per allocare un vettore di dimensione `n`.
2. L'input è dato da una sequenza di numeri terminata da 0. Il vettore sarà ridimensionato dinamicamente: partire da una dimensione fissata (es: 15) e allungare il vettore di una lunghezza fissa (es: 10), ricreandolo, ogni volta che il primo si riempie.